Experiment 10: I2C

Purpose:

Use the PIC16F883 MSSP peripheral for I2C communication. Begin with an address-only master transaction and measurements of the two-wire bus. Later parts add an external sensor, a second PIC operating as a slave, and UART integration.

Standards and References:

- RCET 3375 Lab Standards.
- RCET PIC-AS style Guide and PIC-AS project Setup.
- PIC16F883 Datasheet (DS40Ø0Ø1291H).
- PIC16F88X Silicon Errata
- PICmicro Mid-Range MCU Family Reference Manual (DS33Ø23A).
- AN735: Using the PICmicro MSSP module for master I2C communication
- AN734: Using the PICmicro for slave I2C communication (original).
- AN734: Slave I2C communication (revised).
- TB3252: Using the MSSP in I2C Master Mode.
- AN736: An I2C Network protocal for enviromental monitoring.
- I2C-Bus specification (NXP UM1Ø2Ø4).
- Lab Ø8 UART
- Manufacturer's datasheet for the supplied I2C sensor module.

Equipment and Materials:

Part 1: One PIC16F883 minimum circuit with a 4MHz crystal, 5V supply, PICkit 3, MPLAB X/PIC-AS, single I2C pull-up resistor in common values near the four calculated test target (two of each chosen value, one per bus line), breadboard components, multimeter, and an oscilloscope with I2C decoding.

Later Parts: A second PIC16F883 circuit, supplied I2C sensor breakout and input source, Lab 8 UART host application and serial adapter, ADC sensor and servo components, and optional Mastery hardware.

Experiment 10: I2C

Part 1: I2C Master and Bus Timing

Goal:

Use one PIC16F883 as a 100kHz I2C master. Send a periodic address-only write transaction with no slave connected. Detect the expected NACK, generate STOP, and measure the bus waveforms.

Required Behavior:

- Use a 4MHz crystal and a 5V PIC16F883 circuit. Connect RC3/SCL (Pin 14) and RC4/SDA (Pin 15) to 5V through seperate 4.7kΩ pull-up resistor. Leave the I2C bus without a slave for this part. Configure the pins for MSSP open-drain operation.
- Configure the MSSP for 7-bit I2C master mode at 100kHz. Address 0x20 with R/W = 0. The address and R/W bit form the transmitted byte 0x40 (0100000)
- For every probe, generate START, send the address/R-W byte, read the ACK/NACK results, and generate STOP. With no slave, the 9th clock pulse must show a NACK (SDA High).
- Use a hardware timer to start repeated address transmissions. Choose a timer interval that leaves a clear idle gap between the STOP of one transaction and the START of the next. Calculate the expected interval from your chosen timer settings.
- Use RA0 (pin 2) as a NACK indicator. Clear RA0 at the beginning of each probe; making it High if the slave does not acknowledge. Leave it Low for Ack. Complete the transaction with stop in either case.
- Use RA1 (pin 3) as a timer diagnostic. Set RA1 High when the timer event starts a new transaction, just before calling AddressProbe. Set it low after AddressProbe returns, following STOP. Each RA1 rising edge marks the start of a new transaction and provides an oscilloscope trigger without I2C decoding.

Before Lab:

Prepare the following in the lab book before constructing the circuit.

- A schematic with the PIC power, reset, oscillator, programming connections, seperate SDA/SCL pull-ups, and RA0/RA1 diagnostic connections. Account for every used pin and connection as required by the Lab standard.
- PIC16F883 SFR documentation for MSSP master setup, START/STOP control, address transmission, completion and ACK status, the selected timer's setup and event indication, and the I/O configuration used here. Include the relevent register maps and the values used.
- Flowcharts for the main loop and AddressProbe subroutine. Show the timer event, RA1 diagnostic signal, START/STOP, address transmission, and the NACK desision.

Experiment 10: I2C

Before Lab:

- Predicted SCL clock period, frequency, and the interval between transaction based on the selected timer.
- Pull-up resistor investigation: Starting with the 4.7kΩ resistor, calculate the target resistance for nominal, ·10, ÷2, and ÷10. For each target, select and document a nearby common resistor value. Use one resistor per bus line for each test; do not combine resistors. Record the calculated target, chosen values, resistor ratings, and predicted results.
- For both SDA and SCL at every selected resistance, predict the idle High voltage, current through the pull-up with the bus idle and with the line driven low, and 30% - 70% rise-time. State the assumed bus capacitance and Low voltage. Compare and predict with the I2C standard-mode electrical requirements and PIC16F883 pin limits before applying power.
- Compile the Part 1 source and program the PIC before starting the in the lab steps.

MSSP Timing Example: For a 4MHz oscillator and 100kHz I2C clock, find the master baud-rate value using the datasheet equation:

$f_{SCL} = F_{OSC} / 4(SSPADD+1)$
$SSPADD = (F_{OSC} / 4(F_{SCL})) - 1$
$SSPADD = (4MHz / (4 \cdot 100kHz)) - 1$
$SSPADD = 10 - 1 = 9$
$T_{SCL} = 1/100k = 10\mu S$

Pull-up Voltage and Current Example: With neither SDA nor SCL driven Low, each line should be close to 5V and the ideal pull-up current is 0A. For a Low line, estimate the current through one nominal 4.7kΩ resistor using the assumed 0.4V Low level:

$V_{idle} \approx V_{DD} = 5.0V$
$I_{idle} = V_{DD} - V_{idle} / R_p$
$I_{idle} = 5V - 5V / 4.7K\Omega$
$I_{idle} = 0mA$

$I_{LOW} = V_{DD} - V_{OL} / R_p$
$I_{LOW} = 5V - 0.4V / 4.7K\Omega$
$I_{LOW} = 4.6V / 4.7K\Omega$
$I_{LOW} \approx 0.979mA$

I2C Rise-Time Calculation: Begin with the RC charging equation used in class. Let V_i be the line voltage, V_f be the final voltage, and $\tau = R_p \cdot C_{bus}$ be the time constant:

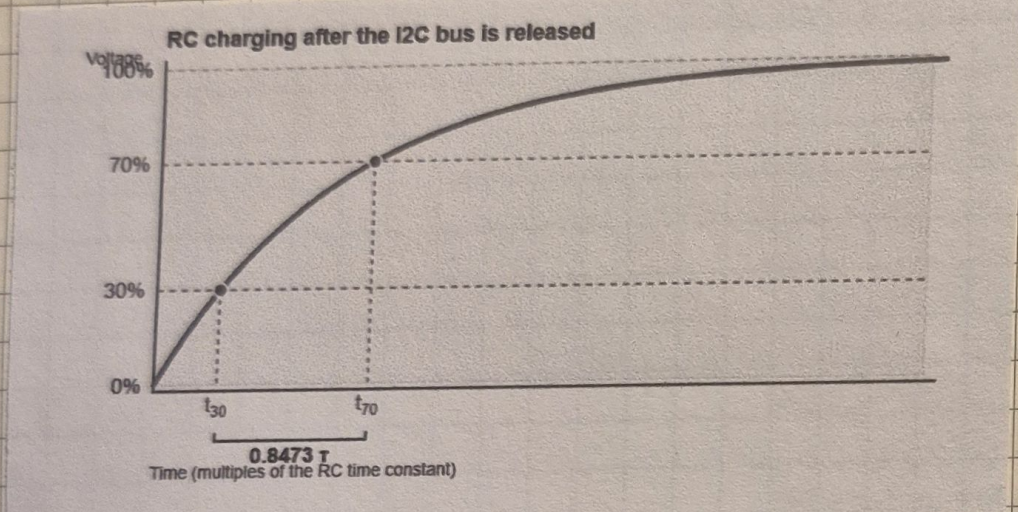
$V_t = V_f - (V_f - V_i) e^{-t/\tau}$
$\tau = R_p \cdot C_{bus}$

Experiment 10: I2C

Before Lab:

For the pull-up estimate, assume the line starts at $V_i = 0V$ and rises towards $V_f = V_{DD}$. Solve the charging equation for the time at which the line reaches a chosen fraction of V_{DD}:

$$V_+ = V_{DD}(1 - e^{-+/\tau})$$
$$V_+/V_{DD} = 1 - e^{-+/\tau}$$
$$e^{-+/\tau} = 1 - V_+/V_{DD}$$
$$+ = -\tau \ln(1 - V_+/V_{DD})$$



I2C rise-time is measured from 30% to 70% of the bus voltage. Calculate the time to reach each threshold, then subtract:

$$t_{30\%} = -\tau \ln(1 - 0.30)$$
$$t_{30\%} = -\tau \ln(0.70)$$
$$\boxed{t_{30\%} = 0.357\tau}$$

$$t_{70\%} = -\tau \ln(1 - 0.70)$$
$$t_{70\%} = -\tau \ln(0.30)$$
$$\boxed{t_{70\%} = 1.204\tau}$$

$$t_r = t_{70\%} - t_{30\%}$$
$$t_r = 1.204 - 0.357$$
$$\boxed{t_r = 0.847\tau}$$

Worked Example: Assume a nominal 4.7kΩ pull-up and 100pF bus capacitance. The capacitance is an example, not measured.

$$\tau = R_p \cdot C_{bus}$$
$$\tau = 4.7k\Omega \cdot 100pF$$
$$\boxed{\tau = 470ns}$$

$$t_r = 0.847\tau$$
$$t_r = 0.847(470ns)$$
$$\boxed{t_r = 398.09ns}$$

Standard I2C specifies a max 1μs rise-time, and its low-level current spec is based on 3mA. The lowest-resistance test can require substantially more low-state current. Check whether each chosen resistor meets the PIC pin current/voltage limits and I2C electrical requirements; explain any out of spec prediction. Do not power a circuit that exceeds the PIC electrical limits. The circuit has wiring, pin, and oscilloscope-probe capacitance even when no slave connected.

~~Required~~ HLC Experiment 10: I2C

Required Setup:

```
MSSP:
    Set up I2C master operation at 100 kHz
    Enable the peripheral for SDA and SCL

Timer:
    Configure a hardware timer for a repeat interval that separates address transactions
    Clear its event indication and start it

I/O:
    Set RA0 as a digital output for the NACK indicator
    Set RA1 as a digital output for the timer diagnostic
    Configure the other used pins for their required functions

Startup:
    Initialize both diagnostic outputs LOW
```

Program Structure:

```
Main:
    Wait for the chosen timer's event
    Clear the timer event indication
    Set RA1 HIGH
    Call AddressProbe
    Set RA1 LOW
    Repeat

AddressProbe:
    Clear the NACK indicator
    Generate a START condition and wait until it completes
    Send the slave address with the write bit and wait until the byte is sent
    Check the acknowledge result
    If the result is NACK, turn on the NACK indicator
    Generate a STOP condition and wait until it completes
    Return
```

Figure 10.1 : Registers Table

TABLE 13-2: REGISTERS ASSOCIATED WITH SPI OPERATION

Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Register on Page
INTCON	GIE/GIEH	PEIE/GIEL	T0IE	INTE	RBIE	T0IF	INTF	RBIF	32
PIE1	—	ADIE	RCIE	TXIE	SSPIE	CCP1IE	TMR2IE	TMR1IE	33
PIR1	—	ADIF	RCIF	TXIF	SSPIF	CCP1IF	TMR2IF	TMR1IF	35
SSPBUF	Synchronous Serial Port Receive Buffer/Transmit Register								179
SSPCON	WCOL	SSPOV	SSPEN	CKP	SSPM3	SSPM2	SSPM1	SSPM0	177
SSPSTAT	SMP	CKE	D/$\overline{A}$	P	S	R/$\overline{W}$	UA	BF	176
TRISA	TRISA7	TRISA6	TRISA5	TRISA4	TRISA3	TRISA2	TRISA1	TRISA0	40
TRISC	TRISC7	TRISC6	TRISC5	TRISC4	TRISC3	TRISC2	TRISC1	TRISC0	54

Legend: x = unknown, u = unchanged, – = unimplemented, read as '0'. Shaded cells are not used by the MSSP in SPI mode.

Note 1: Bit 6 of PORTA, LATA and TRISA are enabled in ECIO and RCIO Oscillator modes only. In all other oscillator modes, they are disabled and read '0'.

Experiment 10: I2C

Figure 10.2: SSP STATUS Register

REGISTER 13-1: SSPSTAT: SSP STATUS REGISTER

R/W-0	R/W-0	R-0	R-0	R-0	R-0	R-0	R-0
SMP	CKE	D/$\overline{A}$	P	S	R/$\overline{W}$	UA	BF
bit 7							bit 0

Legend:			
R = Readable bit	W = Writable bit	U = Unimplemented bit, read as '0'	
-n = Value at POR	'1' = Bit is set	'0' = Bit is cleared	x = Bit is unknown

bit 7 **SMP**: Sample bit

SPI Master mode:
1 = Input data sampled at end of data output time
0 = Input data sampled at middle of data output time

SPI Slave mode:
SMP must be cleared when SPI is used in Slave mode

In I^2C Master or Slave mode:
1 = Slew rate control disabled for standard speed mode (100 kHz and 1 MHz)
0 = Slew rate control enabled for high speed mode (400 kHz)

bit 6 **CKE**: SPI Clock Edge Select bit

CKP = 0:
1 = Data transmitted on falling edge of SCK
0 = Data transmitted on rising edge of SCK

CKP = 1:
1 = Data transmitted on rising edge of SCK
0 = Data transmitted on falling edge of SCK

bit 5 **D/$\overline{A}$**: Data/$\overline{Address}$ bit (I^2C mode only)
1 = Indicates that the last byte received or transmitted was data
0 = Indicates that the last byte received or transmitted was address

bit 4 **P**: Stop bit
(I^2C mode only. This bit is cleared when the MSSP module is disabled, SSPEN is cleared.)
1 = Indicates that a Stop bit has been detected last (this bit is '0' on Reset)
0 = Stop bit was not detected last

bit 3 **S**: Start bit
(I^2C mode only. This bit is cleared when the MSSP module is disabled, SSPEN is cleared.)
1 = Indicates that a Start bit has been detected last (this bit is '0' on Reset)
0 = Start bit was not detected last

bit 2 **R/$\overline{W}$**: Read/Write bit information (I^2C mode only)
This bit holds the R/$\overline{W}$ bit information following the last address match. This bit is only valid from the address match to the next Start bit, Stop bit, or not ACK bit.
In I^2C Slave mode:
1 = Read
0 = Write
In I^2C Master mode:
1 = Transmit is in progress
0 = Transmit is not in progress
OR-ing this bit with SEN, RSEN, PEN, RCEN, or ACKEN will indicate if the MSSP is in Idle mode.

bit 1 **UA**: Update Address bit (10-bit I^2C mode only)
1 = Indicates that the user needs to update the address in the SSPADD register
0 = Address does not need to be updated

bit 0 **BF**: Buffer Full Status bit
Receive (SPI and I^2C modes):
1 = Receive complete, SSPBUF is full
0 = Receive not complete, SSPBUF is empty
Transmit (I^2C mode only):
1 = Data transmit in progress (does not include the $\overline{ACK}$ and Stop bits), SSPBUF is full
0 = Data transmit complete (does not include the $\overline{ACK}$ and Stop bits), SSPBUF is empty

The MSSP module has these six registers for I^2C operation:

- MSSP Control Register 1 (SSPCON)
- MSSP Control Register 2 (SSPCON2)
- MSSP STATUS register (SSPSTAT)
- Serial Receive/Transmit Buffer (SSPBUF)
- MSSP Shift Register (SSPSR) – Not directly accessible
- MSSP Address register (SSPADD)
- MSSP Mask register (SSPMSK)

Experiment 10: I2C

Figure 10.3: SSP Control Register 1

REGISTER 13-2: SSPCON: SSP CONTROL REGISTER 1

R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
WCOL	SSPOV	SSPEN	CKP	SSPM3	SSPM2	SSPM1	SSPM0
bit 7							bit 0

Legend:			
R = Readable bit	W = Writable bit	U = Unimplemented bit, read as '0'	
-n = Value at POR	'1' = Bit is set	'0' = Bit is cleared	x = Bit is unknown

bit 7 **WCOL**: Write Collision Detect bit
Master mode:
1 = A write to the SSPBUF register was attempted while the I^2C conditions were not valid for a transmission to be started
0 = No collision
Slave mode:
1 = The SSPBUF register is written while it is still transmitting the previous word (must be cleared in software)
0 = No collision

bit 6 **SSPOV**: Receive Overflow Indicator bit
In SPI mode:
1 = A new byte is received while the SSPBUF register is still holding the previous data. In case of overflow, the data in SSPSR is lost. Overflow can only occur in Slave mode. In Slave mode, the user must read the SSPBUF, even if only transmitting data, to avoid setting overflow. In Master mode, the overflow bit is not set since each new reception (and transmission) is initiated by writing to the SSPBUF register (must be cleared in software).
0 = No overflow
In I^2C mode:
1 = A byte is received while the SSPBUF register is still holding the previous byte. SSPOV is a "don't care" in Transmit mode (must be cleared in software).
0 = No overflow

bit 5 **SSPEN**: Synchronous Serial Port Enable bit
In both modes, when enabled, these pins must be properly configured as input or output
In SPI mode:
1 = Enables serial port and configures SCK, SDO, SDI and $\overline{SS}$ as the source of the serial port pins
0 = Disables serial port and configures these pins as I/O port pins
In I^2C mode:
1 = Enables the serial port and configures the SDA and SCL pins as the source of the serial port pins
0 = Disables serial port and configures these pins as I/O port pins

bit 4 **CKP**: Clock Polarity Select bit
In SPI mode:
1 = Idle state for clock is a high level
0 = Idle state for clock is a low level
In I^2C Slave mode:
SCK release control
1 = Release clock
0 = Holds clock low (clock stretch). (Used to ensure data setup time.)
In I^2C Master mode:
Unused in this mode

bit 3-0 **SSPM<3:0>**: Synchronous Serial Port Mode Select bits
0000 = SPI Master mode, clock = Fosc/4
0001 = SPI Master mode, clock = Fosc/16
0010 = SPI Master mode, clock = Fosc/64
0011 = SPI Master mode, clock = TMR2 output/2
0100 = SPI Slave mode, clock = SCK pin, $\overline{SS}$ pin control enabled
0101 = SPI Slave mode, clock = SCK pin, $\overline{SS}$ pin control disabled, $\overline{SS}$ can be used as I/O pin
0110 = I^2C Slave mode, 7-bit address
0111 = I^2C Slave mode, 10-bit address
1000 = I^2C Master mode, clock = Fosc / (4 * (SSPADD+1))
1001 = Load Mask function
1010 = Reserved
1011 = I^2C firmware controlled Master mode (Slave idle)
1100 = Reserved
1101 = Reserved
1110 = I^2C Slave mode, 7-bit address with Start and Stop bit interrupts enabled
1111 = I^2C Slave mode, 10-bit address with Start and Stop bit interrupts enabled

The Baud Rate Generator used for the SPI mode operation is now used to set the SCL clock frequency for either 100 kHz, 400 kHz, or 1 MHz I^2C operation. The Baud Rate Generator reload value is contained in the lower seven bits of the SSPADD register. The Baud Rate Generator will automatically begin counting on a write to the SSPBUF. Once the given operation is complete (i.e., transmission of the last data bit is followed by ACK), the internal clock will automatically stop counting and the SCL pin will remain in its last state.

Experiment 10: I2C

Figure 10.4: SSP control Register 2

REGISTER 13-3: SSPCON2: SSP CONTROL REGISTER 2

R/W-0	R-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
GCEN	ACKSTAT	ACKDT	ACKEN	RCEN	PEN	RSEN	SEN
bit 7							bit 0

Legend:			
R = Readable bit	W = Writable bit	U = Unimplemented bit, read as '0'	
-n = Value at POR	'1' = Bit is set	'0' = Bit is cleared	x = Bit is unknown

bit 7 **GCEN:** General Call Enable bit (in I^2C Slave mode only)
1 = Enable interrupt when a general call address (0000h) is received in the SSPSR
0 = General call address disabled

bit 6 **ACKSTAT:** Acknowledge Status bit (in I^2C Master mode only)
In Master Transmit mode:
1 = Acknowledge was not received from slave
0 = Acknowledge was received from slave

bit 5 **ACKDT:** Acknowledge Data bit (in I^2C Master mode only)
In Master Receive mode:
Value transmitted when the user initiates an Acknowledge sequence at the end of a receive
1 = Not Acknowledge
0 = Acknowledge

bit 4 **ACKEN:** Acknowledge Sequence Enable bit (in I^2C Master mode only)
In Master Receive mode:
1 = Initiate Acknowledge sequence on SDA and SCL pins, and transmit ACKDT data bit. Automatically cleared by hardware.
0 = Acknowledge sequence idle

bit 3 **RCEN:** Receive Enable bit (in I^2C Master mode only)
1 = Enables Receive mode for I^2C
0 = Receive idle

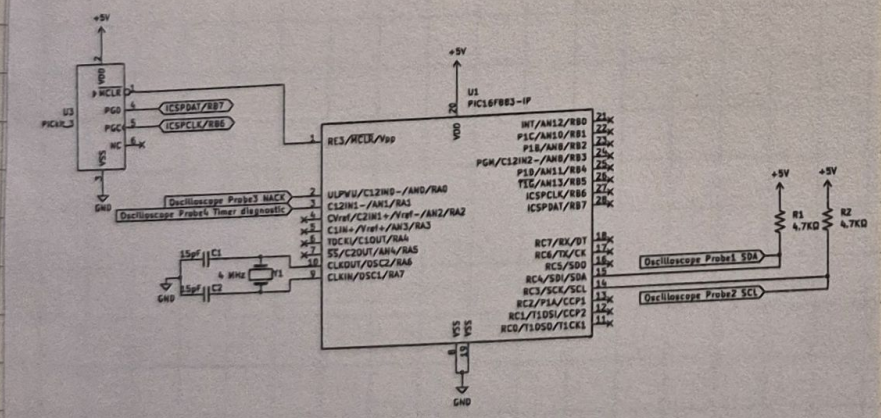
bit 2 **PEN:** Stop Condition Enable bit (in I^2C Master mode only)
SCK Release Control:
1 = Initiate Stop condition on SDA and SCL pins. Automatically cleared by hardware.
0 = Stop condition Idle

bit 1 **RSEN:** Repeated Start Condition Enabled bit (in I^2C Master mode only)
1 = Initiate Repeated Start condition on SDA and SCL pins. Automatically cleared by hardware.
0 = Repeated Start condition Idle

bit 0 **SEN:** Start Condition Enabled bit (in I^2C Master mode only)
In Master mode:
1 = Initiate Start condition on SDA and SCL pins. Automatically cleared by hardware.
0 = Start condition Idle
In Slave mode:
1 = Clock stretching is enabled for both slave transmit and slave receive (stretch enabled)
0 = Clock stretching is disabled

Note 1: For bits ACKEN, RCEN, PEN, RSEN, SEN: If the I^2C module is not in the Idle mode, this bit may not be set (no spooling) and the SSPBUF may not be written (or writes to the SSPBUF are disabled).

Figure 10.5: Schematic



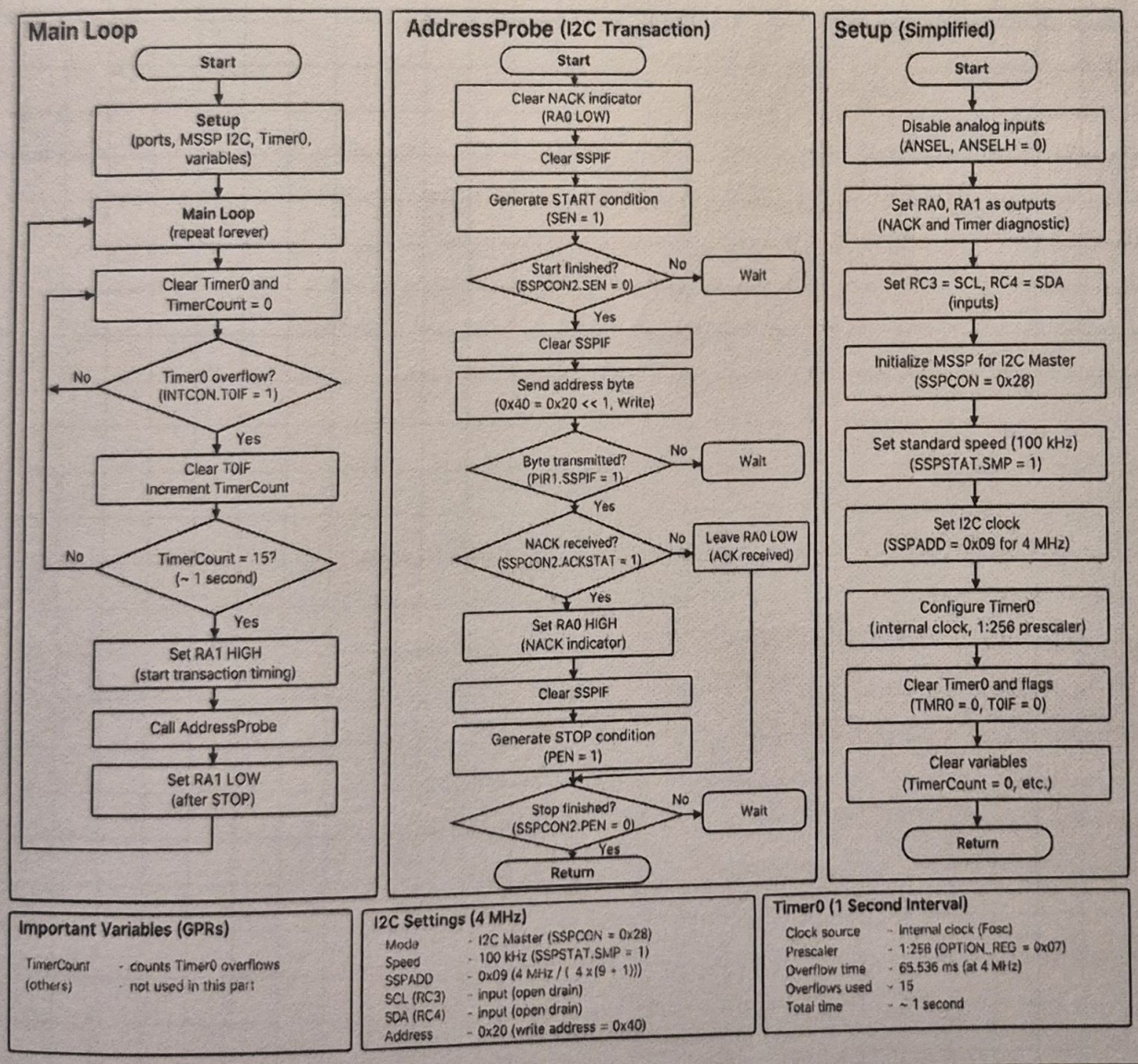
Experiment 10: I2C

Table 10.1: Resistor Prediction

Calculated Target	*Standard Value*	*Resistor Rating*	*Predicted Electrical Results*	*Idle HIGH*	*Idle Current*	*LOW Current*	*Rise Time*
4.7 kΩ	4.7 kΩ	500mW	398.09 ns	5V	0mA	0.979 mA	0.398 µs
47 kΩ	47 kΩ	500mW	3.981 us	5V	0mA	0.098 mA	3.982 µs
2.35 kΩ	2.2 kΩ	500mW	186.34 ns	5V	0mA	2.091 mA	0.186 µs
470 Ω	470 Ω	500mW	39.809 ns	5V	0mA	9.787 mA	0.040 µs

The PIC16F883 datasheet spec for max output current for sink/source is 25mA, making each of the resistors in spec. However, the 47 kΩ resistor has the problem that it draws less current and produces a much slower rising edge, which exceeds the Standard-mode timing limit under the 100 pF assumption. The 470 Ω resistor finds the problem of power, which can be less effective when using things like batteries.

Figure 10.6: Flowcharts

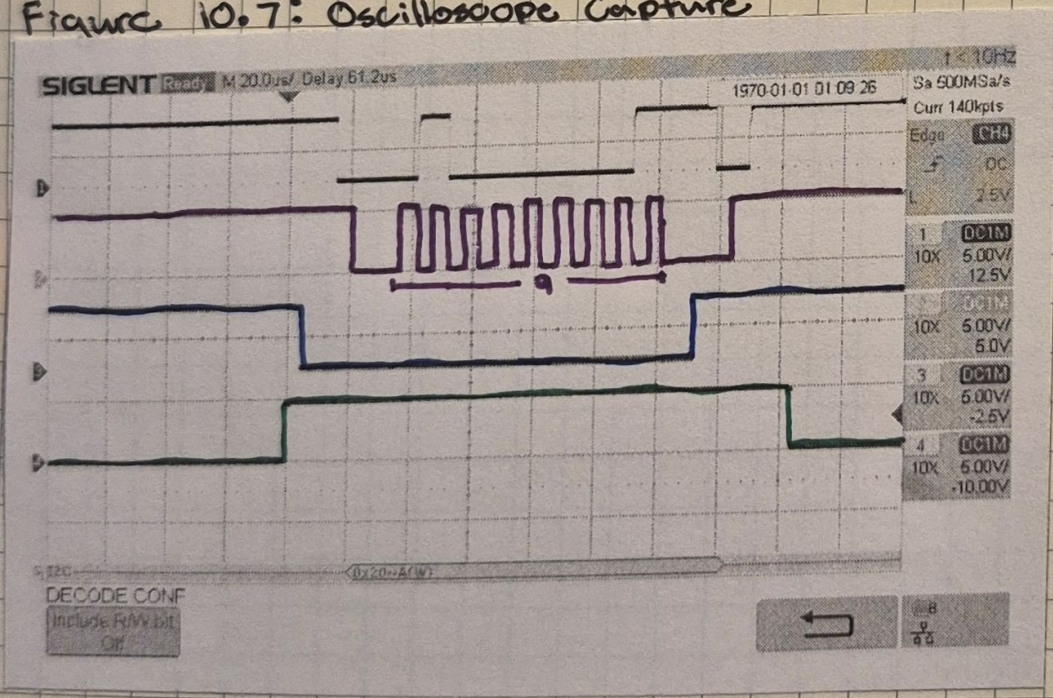


Experiment 10: I2C

In The Lab:

1.) Build the circuit. Check the supply setting, ground, SDA/SCL pull-ups, and connections before applying power, then measure 5V at the PIC. Scan the bus without a slave.
2.) Connect Ch1 = SDA, Ch2 = SCL, Ch3 = RA0 (NACK), and Ch4 = RA1 (Timer Diagnostic). Trigger on the rising edge of Ch4.
3.) Measure the interval between RA1 rising-edges and compare it with your calculated timer interval.
4.) With the power off, remove one pull-up at a time and observe the effect. Restore both resistors and verify normal operation.
5.) Configure 7-bit I2C decoding and capture START, the 7 address bits (0x20), R/W = 0, the ninth-clock NACK, and STOP. Verify RA0 indicates NACK. Note if your decoder displays the 8-bit address/R-W byte (0x40) instead and verify oscilloscope decode setup. Verify that RA1 rises before START and falls after STOP, with a clear idle gap between transactions.
6.) With nominal pull-up installed, measure the SCL period/frequency, clock High/Low times, SDA/SCL rise times (30% - 70%), idle High, and driven Low voltage.
7.) Repeat the measurements for nominal, ·10, ÷2, ÷10 resistances. Use one selected standard resistor per line, with no combinations. Power off before changing resistors and measure each resistor's actual value. Check electrical limits before powering each configuration. Calculate pull-up currents from measured voltages and resistances. Compare predicted and measured values for both lines; note if idle current is below measurement resolution.

Figure 10.7: Oscilloscope Capture



Ch1: SDA

Ch2: SCL

Ch3: RA0 (Nack)

Ch4: RA1 (Timer Diagnostic)

The oscilloscope capture shows the PIC successfully generating a START, the I2C address (0x20), detecting a NACK because no slave is connected (RA0 goes High). RA1 goes High during the transaction and low when it finishes. The address 0x20 appears as 0x40 with the R/W bit included.

10-27-25

Experiment 10: I2C

Part 2: I2C Master with an ADS1115 Sensor Slave

Goal:

Use the Part 1 I2C master to identify an ADS1115, inspect and change it's configuration, and read its ADC results. Observe the conversion-ready signal to choose a suitable reading interval.

Before Lab:

- Bring your working Part 1 address-probe, compiled and programmed into the PIC.
- Skim th ADS1115 datasheet. Find the pin descriptions, supply/input limits, and I2C address selection. You will investigate the registers and configure the ADC as you work through the lab.

Figure 10.8: ADS1115 Pinout and Block Diagram

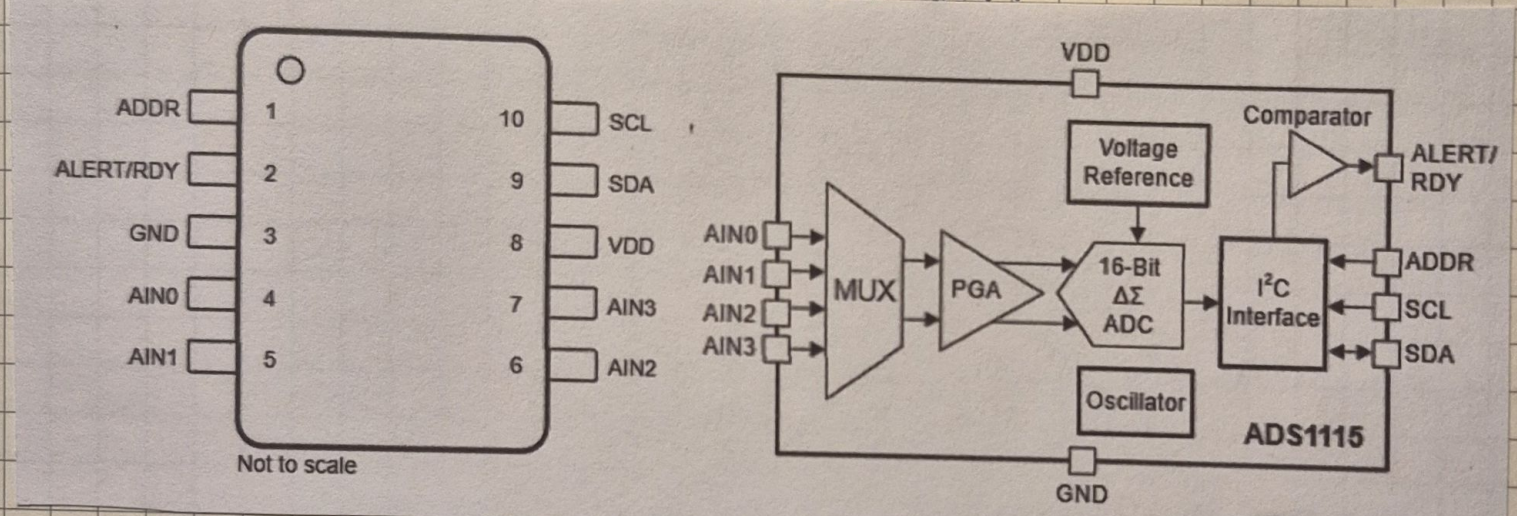


Figure 10.9: ADS1115 Absolute Max Ratings

		MIN	MAX	UNIT
Power-supply voltage	VDD to GND	–0.3	7	V
Analog input voltage	AIN0, AIN1, AIN2, AIN3	GND – 0.3	VDD + 0.3	V
Digital input voltage	SDA, SCL, ADDR, ALERT/RDY	GND – 0.3	5.5	V
Input current, continuous	Any pin except power supply pins	–10	10	mA
Temperature	Operating ambient, T_A	–40	125	°C
	Junction, T_J	–40	150	
	Storage, T_{stg}	–60	150	

Experiment 10: I2C

Before Lab:

Figure 10.10: Target Addresses

ADDR PIN CONNECTION	TARGET ADDRESS
GND	1001000b
VDD	1001001b
SDA	1001010b
SCL	1001011b

Figure 10.11: Updated Master/Slave Schematic

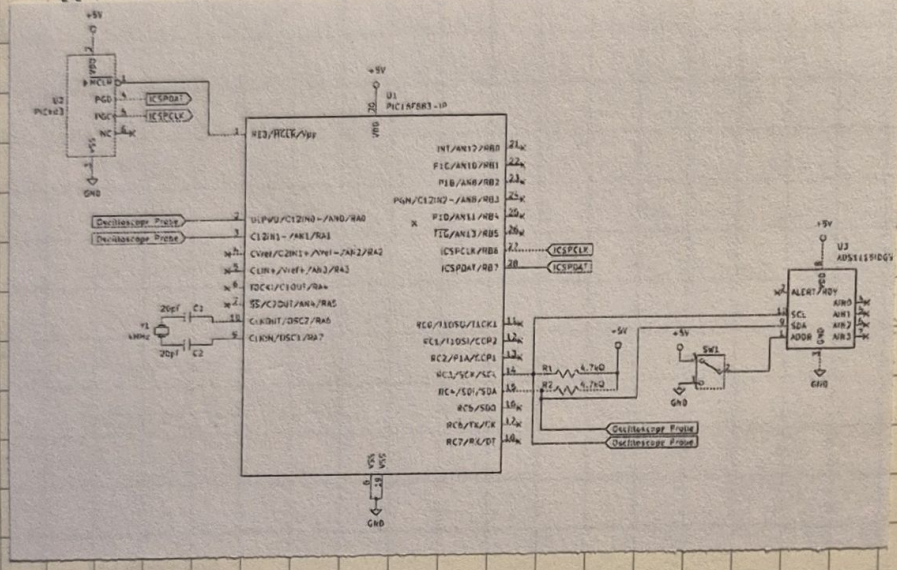
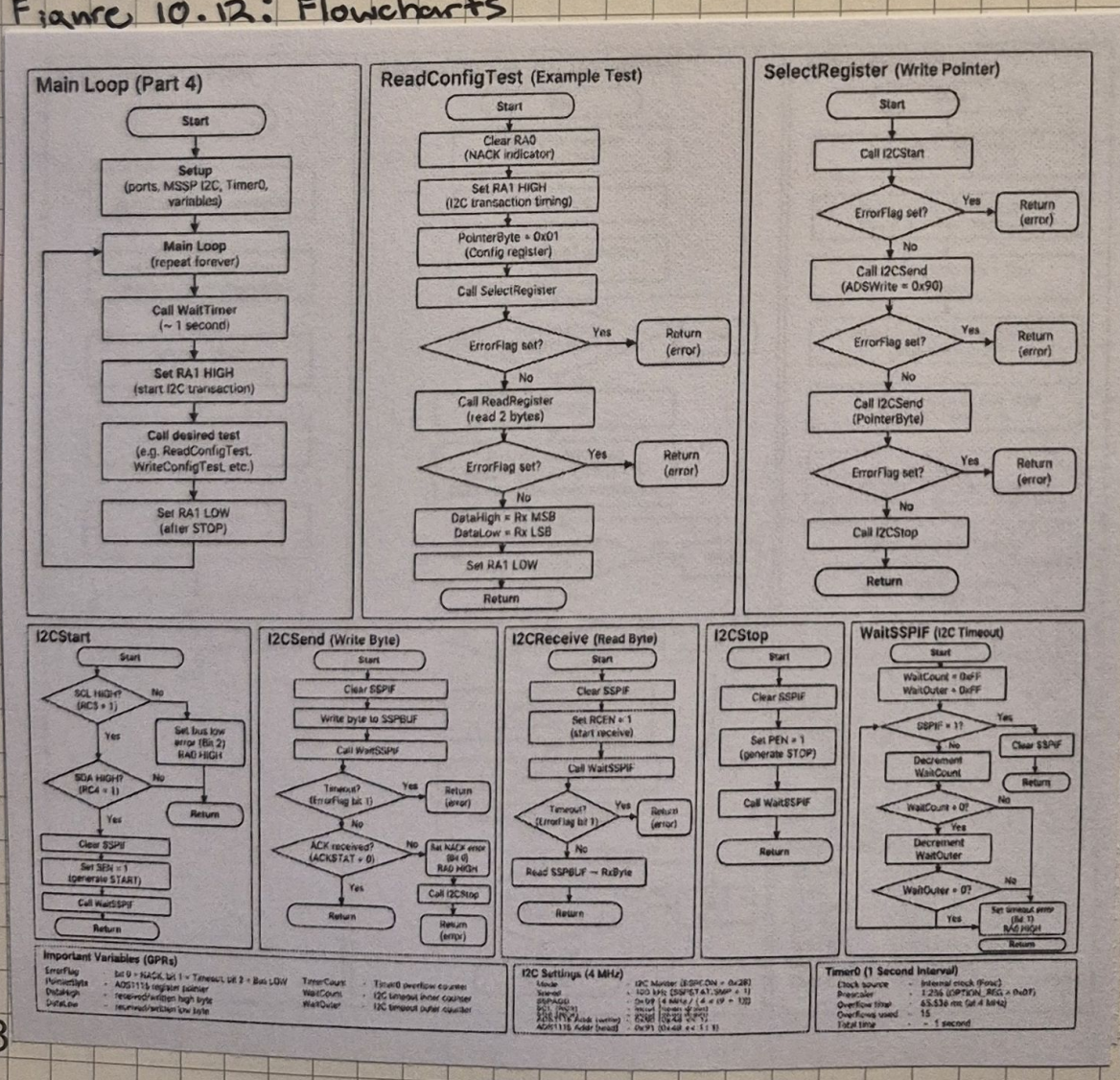


Figure 10.12: Flowcharts



Experiment 10: I2C

In The Lab:

1.) Find the ADS1115 address. Connect the breakout to the Part 1 SDA/SCL bus. Before applying power, check the board's supply power and I2C logic levels against the PIC circuit. Document the connections in your schematic. Find the four possible 7-bit ADS1115 addresses in the datasheet. With ADDR tied to GND, use the Part 1 probe to try all four addresses recording ACK or NACK for each:

Table 10.2: 7-bit Address ACK/NACK Measurements

7-bit address from data sheet	Transmitted write byte	ADDR = GND	ADDR = VDD
1001000b	0	ACK	NACK
1001001b	0	NACK	ACK
1001010b	0	NACK	NACK
1001011b	0	NACK	NACK

The 7-bit addresses "1001010b" and "1001011b" require to be connected to the SDA and SDL respectively in order to be seen as an ACK.

```
For each candidate: START → [address + Write bit] → ACK or NACK → STOP
```

2.) Read the config register. Find the ADS1115 register-pointer table. The one-byte pointer 0x01 selects Config; It does not overwrite Config. On a freshly powered ADS1115, the expected Config value is 0x8583. Select the pointer, then read the 16-bit register with a new start.

```
Select Config (once):
START → ADDR+W → ACK → 0x01 → ACK → STOP

Read Config (once):
START → ADDR+R → ACK → Config MSB → PIC ACK → Config LSB → PIC NACK → STOP
```

Run the pointer write and config read once after MCLR reset, then enter a do nothing loop. Arm the oscilloscope in single sequence mode before releasing MCLR. Combine the two bytes and explain the observed Config fields using the datasheet. MCLR resets the PIC, not the ADS1115; to check the ADS1115's power-up value, power-cycle the ADC before this test. The ACKs after address/pointer bytes come from the ADS1115; PIC ACK and PIC NACK are sent by the master after received bytes.

Experiment 10: I2C

In The Lab:

3.) Configure the ADC. Now use the config register descriptions to determine the two bytes required for:

- AIN0 relative to GND
- ± 6.144 V full-scale Range
- Continuous conversation mode
- 128 samples per second
- Comparator disabled

Document each chosen field and derive the complete 16-bit value. Write using this:

```
Configure ADC (once at startup):
START → ADDR+W → ACK → 0x01 → ACK → [Config MSB] → ACK → [Config LSB] → ACK → STOP
```

4.) Read ADC conversation. Connect a potentiometer across ADS1115 VDD and GND, with it's wiper at AIN0. Do not exceed the ADC's supply voltage and it's analog input. The ~~analog~~ the ADS1115 now converts continuously on it's own. The PIC reads the latest completed result only when it requests it. If the ADS1115 has lost power since step 3, configure it once before starting the read loop. Start with the timer interval from part 1; you will choose a final interval in step 5. Each PIC polling cycle repeats both transactions:

```
Select Conversion:
START → ADDR+W → ACK → 0x00 → ACK → STOP

Read Conversion:
START → ADDR+R → ACK → ADC MSB → PIC ACK → ADC LSB → PIC NACK → STOP

Repeat both transactions at the PIC timer interval.
```

The ADS1115 retains it's register pointers, but for this lab send pointer 0x00 on every polling ~~event~~ cycle. If an address or write-data byte is NACKed, stop sending data, attempt stop, and preserve/report the error. On a timeout or bus-low error, repeat the failure and do not assume stop or bus recovery will succeed; do not continue the transaction. The master's NACK after the final receive byte is normal. Combine the ADC bytes as a signed 16-bit result. For the assigned full-scale range:

$$ADC_{16} = (MSB << 8) \mid LSB$$
$$V_{AIN0} = (\text{signed } ADC_{16} / 32768) \cdot 6.144V$$

For example, a positive result of 0x2000 = 8192 counts given:

$$V_{AIN0} = (8192 / 32768) \cdot 6.144V = 1.536V$$

the ± 6.144V range determines ADC scaling; it does not raise the allowable input voltage above ADS1115 VDD.

Experiment 10: I2C

In The Lab:

5.) Observe ALERT/RDY and choose a polling interval. Find Conversation Ready PIN in the ADS1115 datasheet. Configure ALERT/RDY for conversation-ready pulses using the threshold registers and config fields. The output is open-drain; ensure it has a pull-up to a compatible supply.

```
Set low threshold (once):
START → ADDR+W → ACK → 0x02 → ACK → [Lo MSB] → ACK → [Lo LSB] → ACK → STOP

Set high threshold (once):
START → ADDR+W → ACK → 0x03 → ACK → [Hi MSB] → ACK → [Hi LSB] → ACK → STOP

Enable conversion-ready output (once):
START → ADDR+W → ACK → 0x01 → ACK → [Updated Config MSB] → ACK → [Updated Config LSB] → ACK → STOP
```

6.) Capture continuous reading. With your selected PIC timer interval, capture multiple consecutive conversion-pointer/write and conversion-read transaction at each potentiometer extreme. Measure AIN0 with a DMM at each setting and compare the measured voltage with the ADC result. Use Ch1 = SDA, Ch2 = SCL for decoding; connect another scope channel to ALERT/RDY for the timing test.

Figure 10.13: Find ADS1115 0x48 ACK Capture

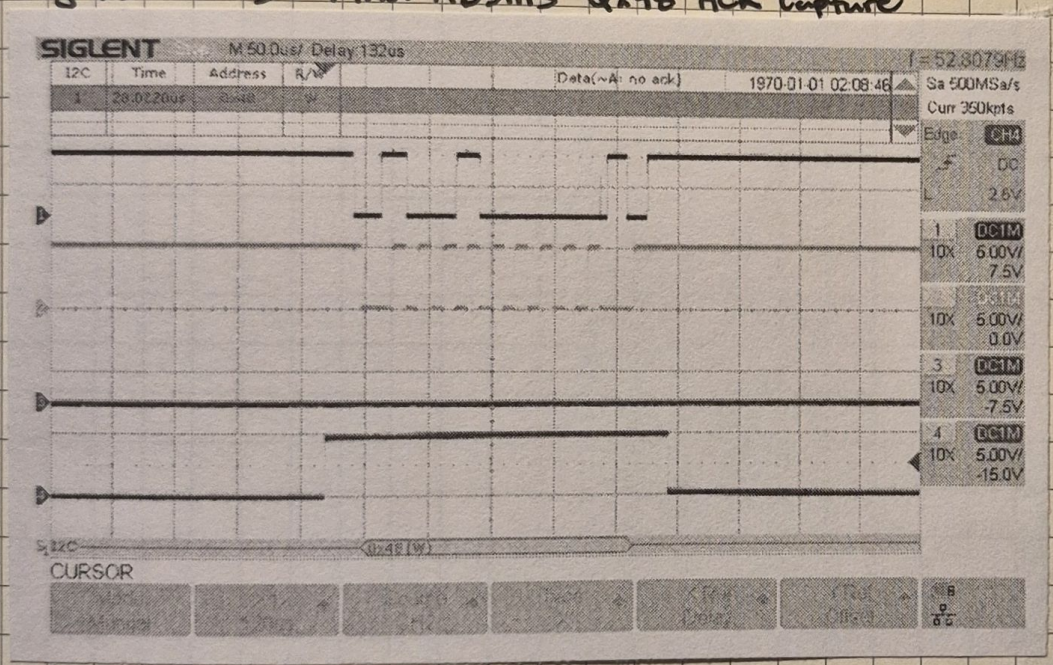
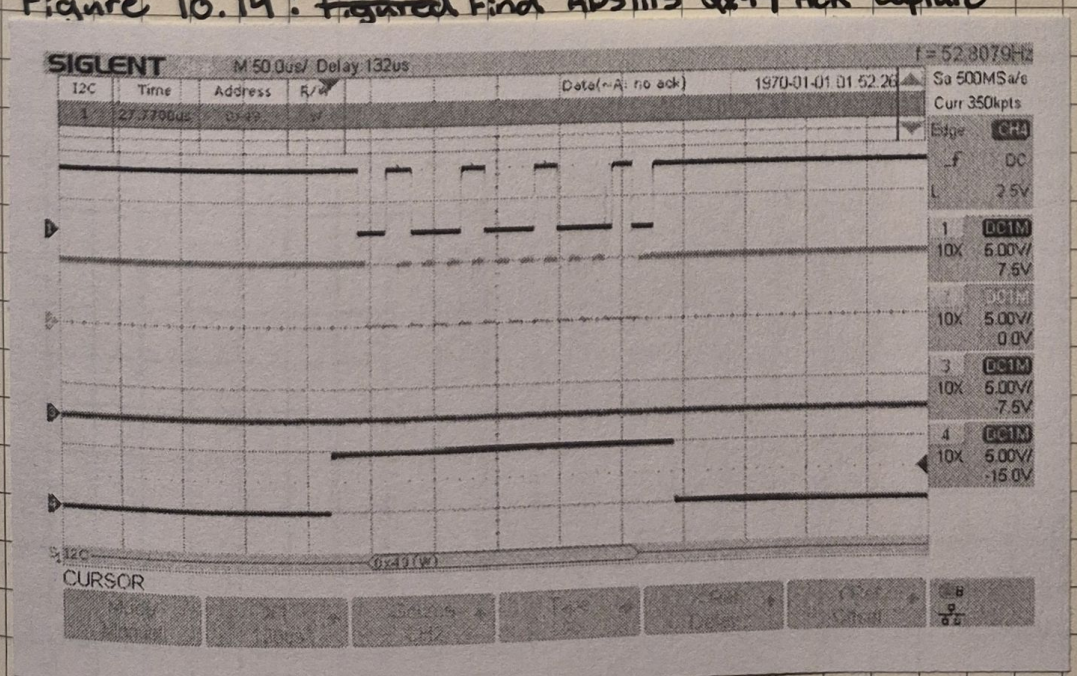


Figure 10.14: ~~Figured~~ Find ADS1115 0x49 Ack Capture



Experiment 10: I2C

In The Lab:

Figure 10.15: Config Read Capture

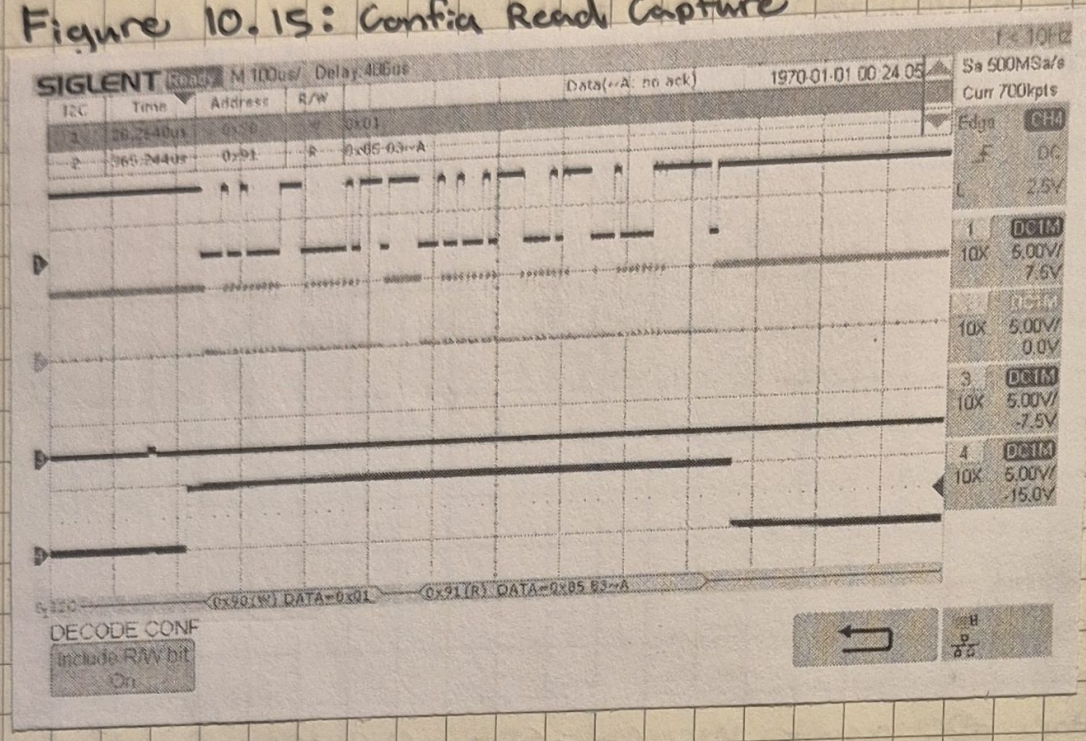


Figure 10.16: Configured ADS1115 Capture

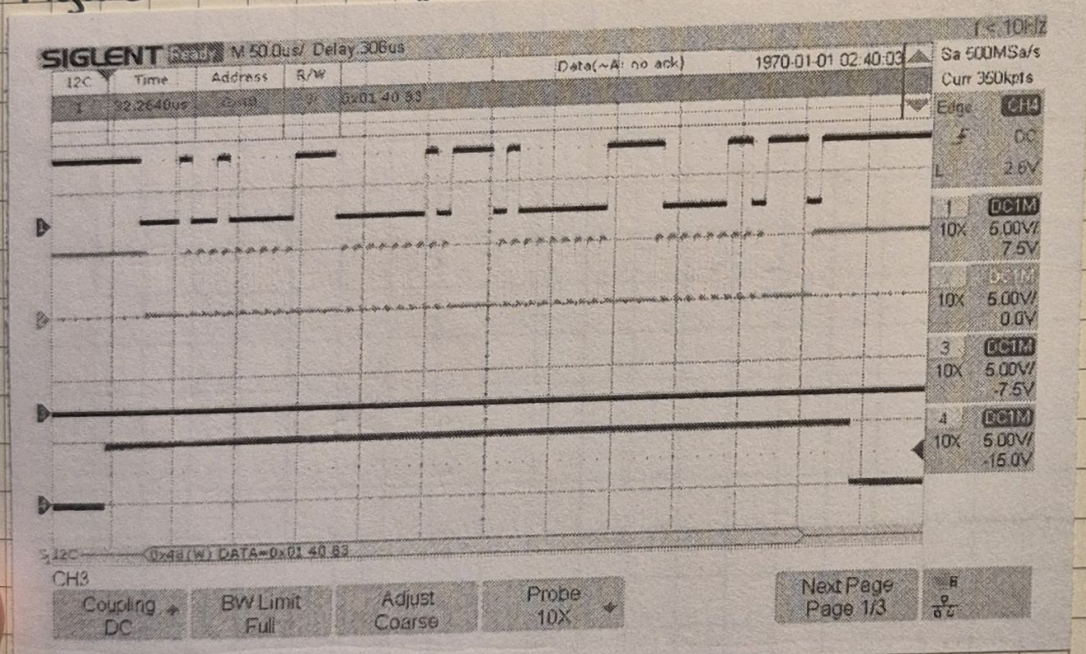


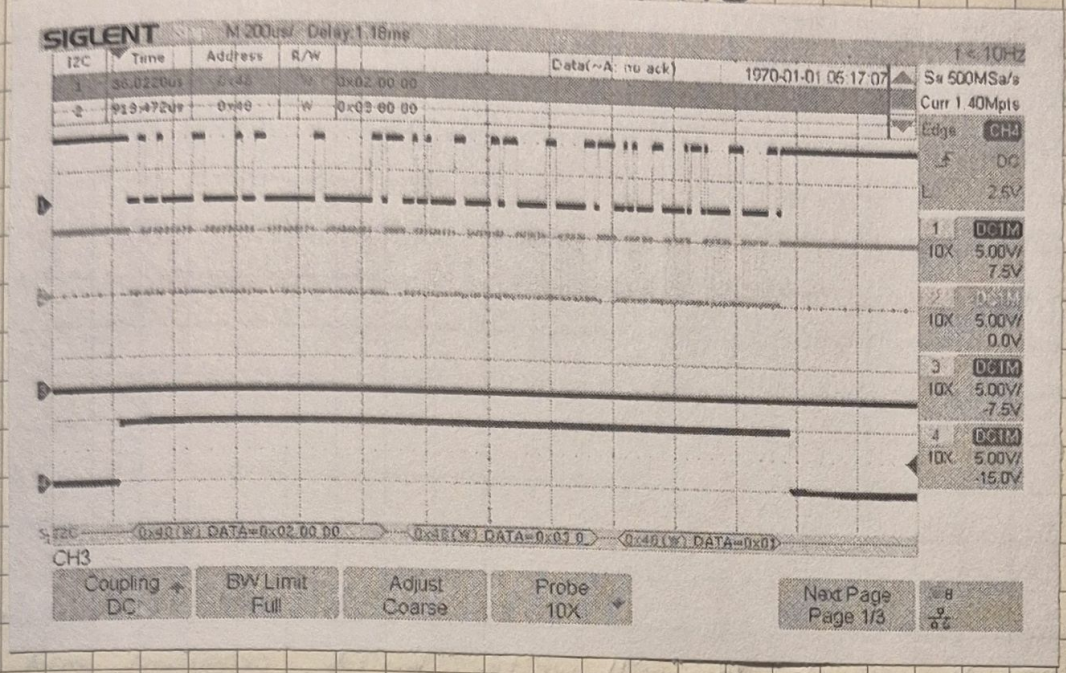
Table 10.3: Read ADC Measurements

Measurement	*Low Pot*	*High Pot*
DMM voltage	0 V	4.96 V
Combined signed ADC	0xFFF2	0x67FE
Calculated voltage	0 V	4.992 V
Difference from DMM	0 V	0.32 V

Experiment 10: I2C

In The Lab:

Figure 10.17: ALERT/RDY Capture



TJR
10-29-26

Table 10.4: Continuous Reading Data

I2C	*Time*	*Address*	*R/W*	*Data*
1	4.80000us	0x48	W	0x02 00 00
2	596.800us	0x48	W	0x03 80 00
3	1.19000ms	0x48	W	0x01 40 80
4	984.778ms	0x48	W	0x00
5	985.114ms	0x48	R	0x13 36~A
6	1.96863s	0x48	W	0x00
7	1.96897s	0x48	R	0x13 37~A

Experiment 10: I2C

Part 3: PIC I2C Slave: Input, Display, and ADC

Goal:

Part 3 requires two PIC16F883 programs: I2C master code and I2C slave code, running on seperate PICs. Develop the slave at 7-bit address 0x20 and extend your Part 2 master code to test the slave, read its digital inputs, write display characters, and retrieve 10-bit ADC measurements. The slave uses a small register pointer interface similar in concept to the AD5113. The slave must normally service I2C events promptly. Either interrupt or polling the MSSP event flag is acceptable. Keep hardware communication routines reusable and seperate from application behavior.

Before Lab:

- Bring your working Part1/Part2 master source code. For Part 3, create a master MPLab and a seperate slave MPLab project. You will write, build, program, and test both master and slave firmware throughout Part 3. Select the correct Main before programming each PIC.
- Draw the complete two-PIC system schematic: RC3/SCL (pin 14), RC4/SDA (Pin 15), common ground, and the effective SDA/SCL pull-ups. Account for pull-ups already installed on connected boards; do not assume every device needs another pair.
- Use the PIC16F883 datasheet to document MSSP 7-bit slave-mode setup, address matching, SSPBUF, ACK/NACK, receive/transmit events, SSPIF, CLKP/clock-stretching, SSPOV and WCOL. Explain which device controls the final ACK/NACK of a read and when the slave can hold SCL low.
- Prepare flowcharts for slave initialization and servicing a I2C event. Include receive-data, transmit-data, and error paths. Extend these flowcharts

Suggested Software Organization (Optinal):

One approach is to seperate the slave code into level 1 (MSSP hardware events), Level 2 (pointer and read/write transactions), and Level 3 (PortB, display, ADC). The master code can reuse your Part 2 I2C routines, ~~which~~ like with a new application logic to send each test transaction and receive its results. This is a suggested design, not a file layout. Each PIC project may use PIC-AS .S file or multiple .S modules if responsibilities are clear. Do not put executable routine in .inc files.

Experiment 10: I2C

Before Lab:

Figure 10.18: PIC Master/Slave schematic

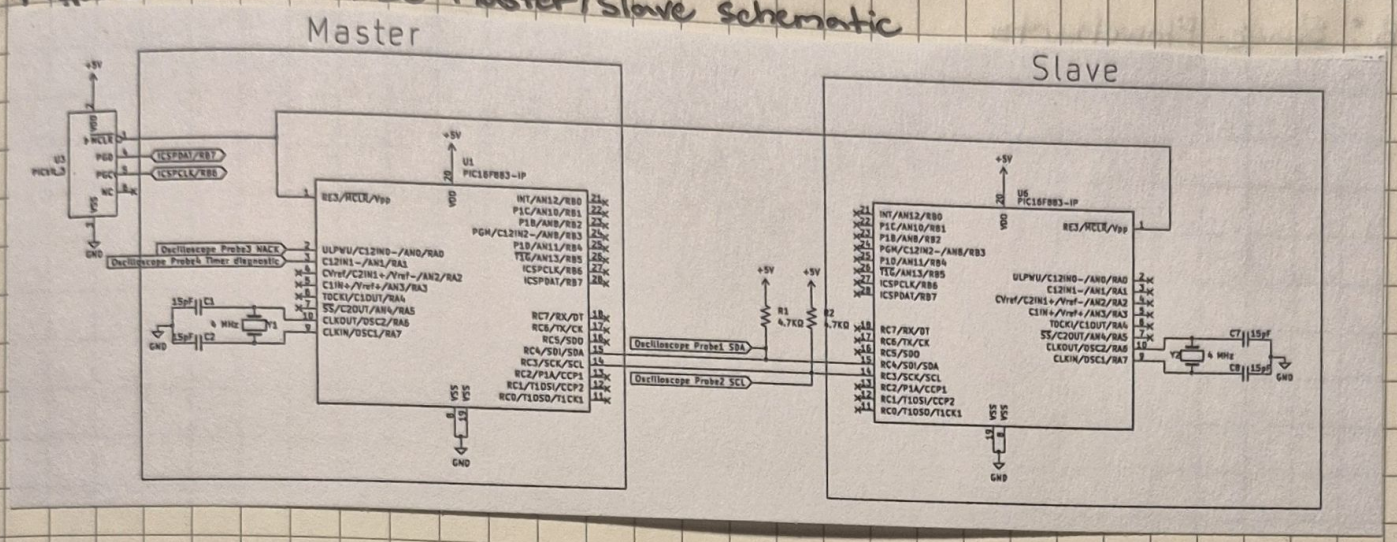
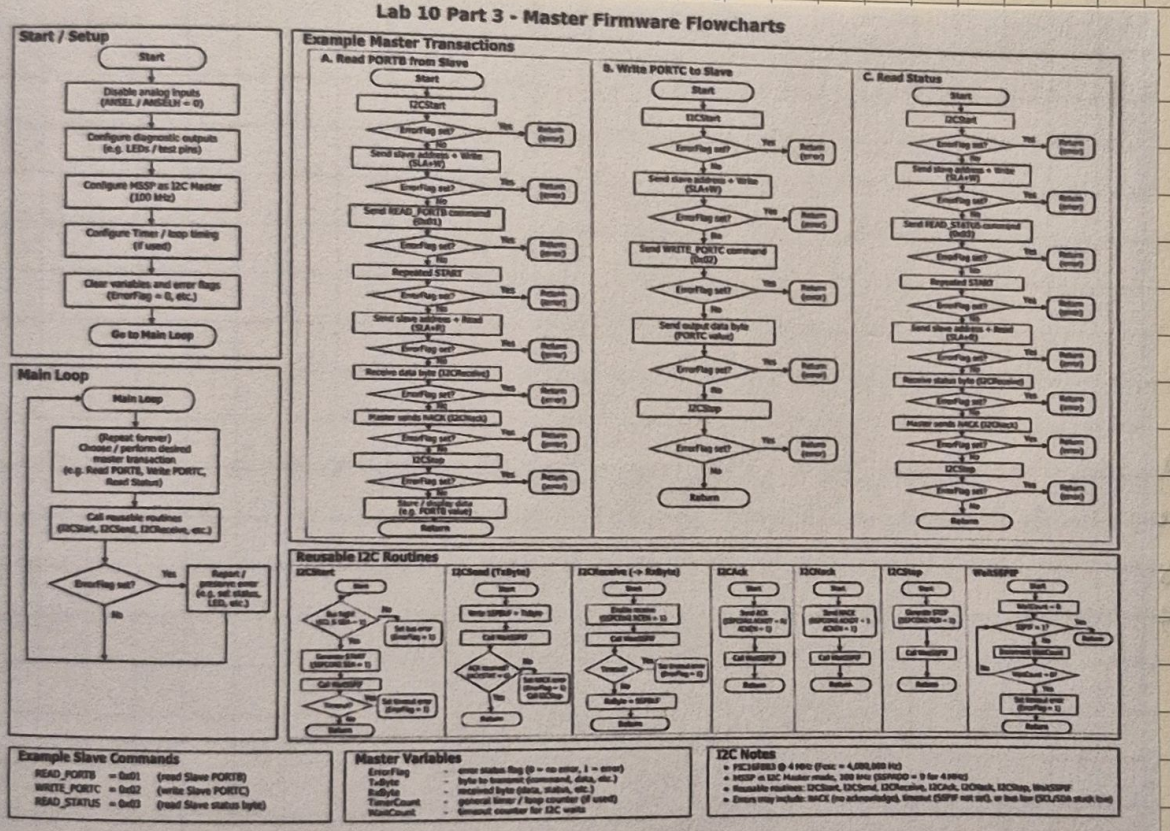


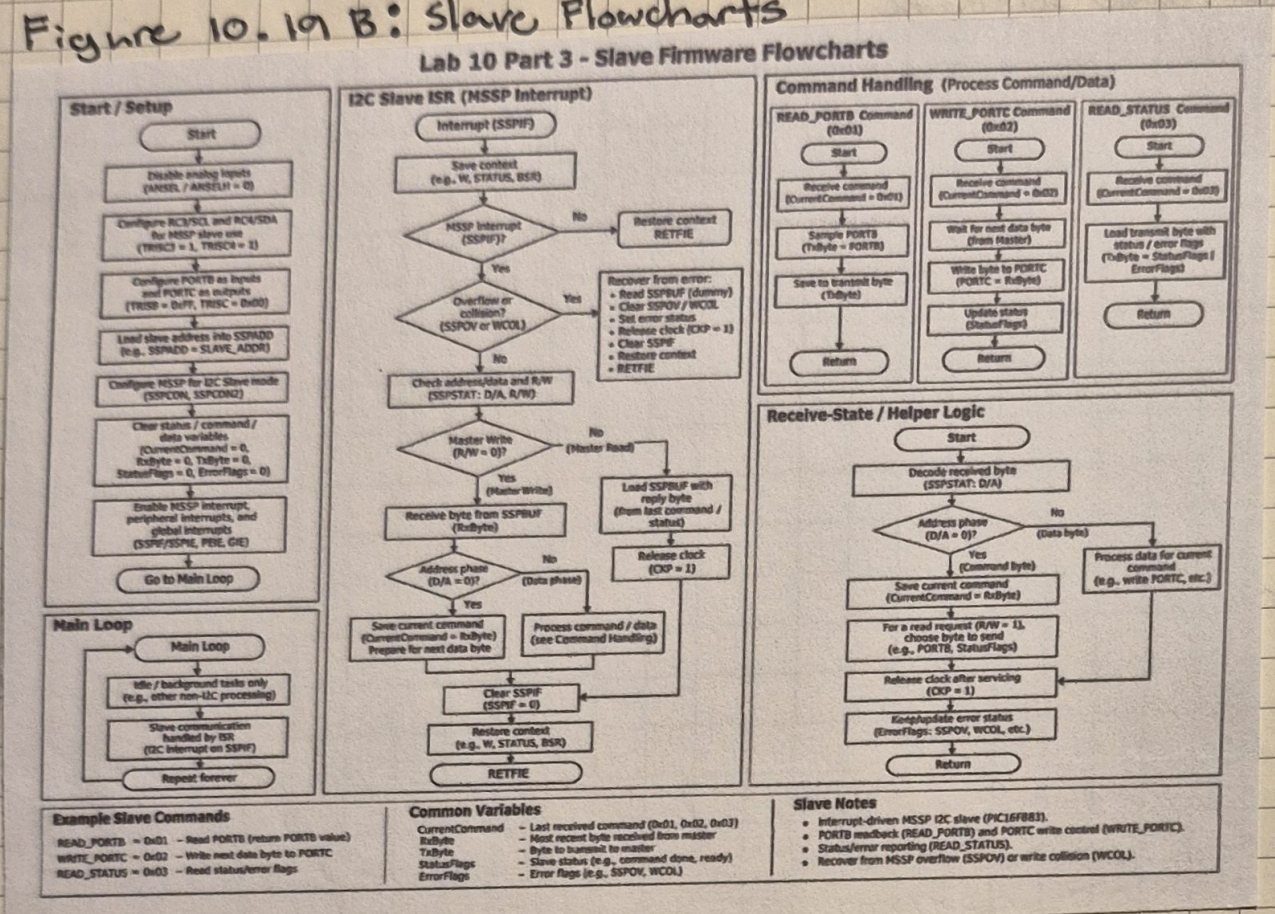
Figure 10.19A: Master Flowcharts



Experiment 10: I2C

Before Lab:

Figure 10.19 B: Slave Flowcharts

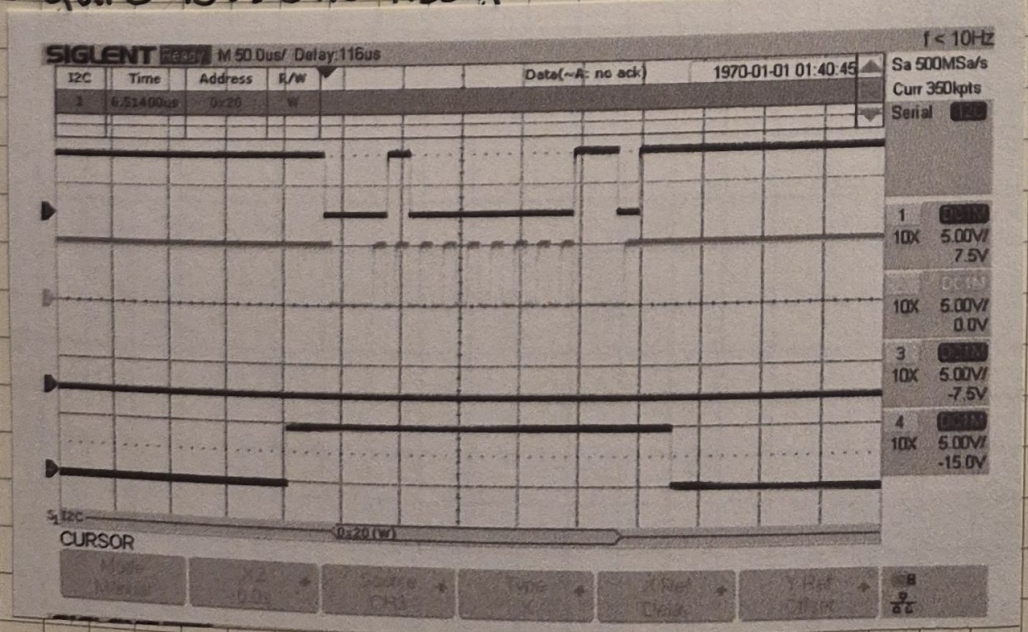


In The Lab:

1.) Verify the slave address. Assemble the minimum slave circuit and connect it to the existing master SDA/SCL bus. Confirm the slave PIC is programmed and powered. Configure its MSSP for the 7-bit address 0x20. Use the Part 1 master address probe, first with the assigned address and then with an unrelated address:

```
START → 0x20 + W (0x40) → slave ACK → STOP
START → other address + W → NACK → STOP
```

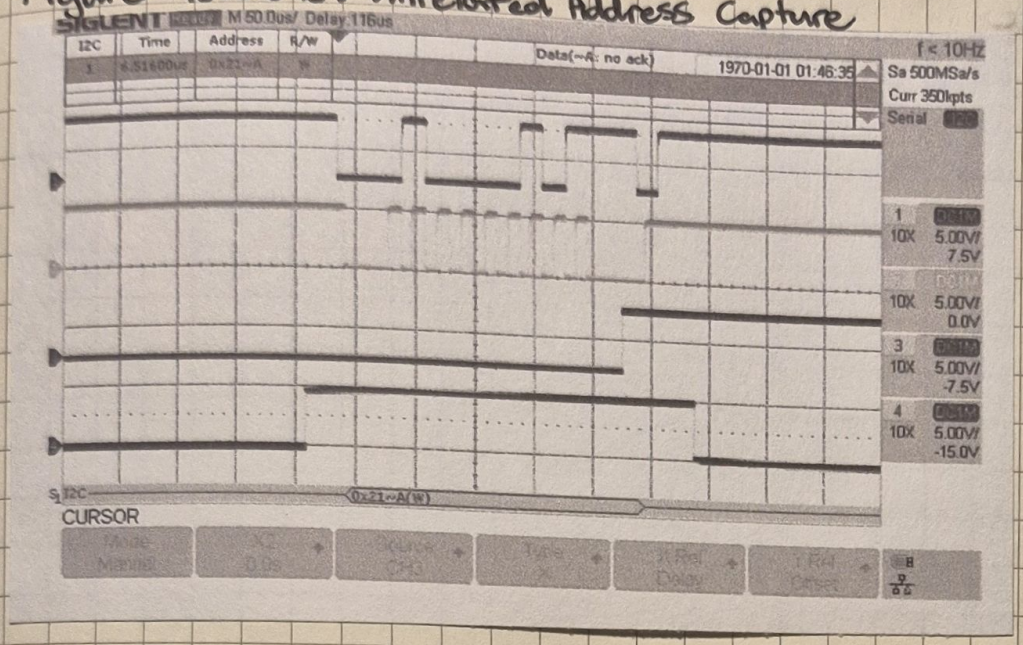
Figure 10.20 A: Assigned Address Capture



Experiment 10: I2C

In The Lab:

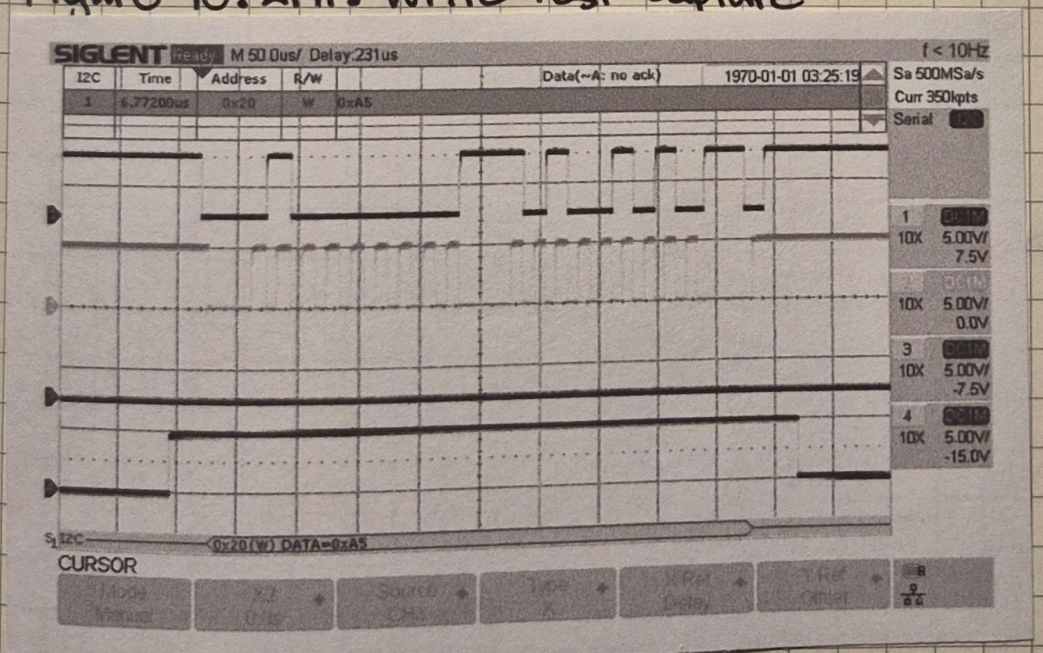
Figure 10.20B: Unrelated Address Capture



2.) Test one received byte and one transmitted byte. Extend the slave to accept a byte after it's address and modify the master to perform both tests. First write 0xA5 and check both slave ACKs. Next, have the slave return a fixed byte 0x5A to a master read. For the one-byte read, the master sends NACK after receiving the byte.

```
Write test: START → 0x40 → ACK → 0xA5 → ACK → STOP
Read test:  START → 0x41 → ACK → 0x5A → Master NACK → STOP
```

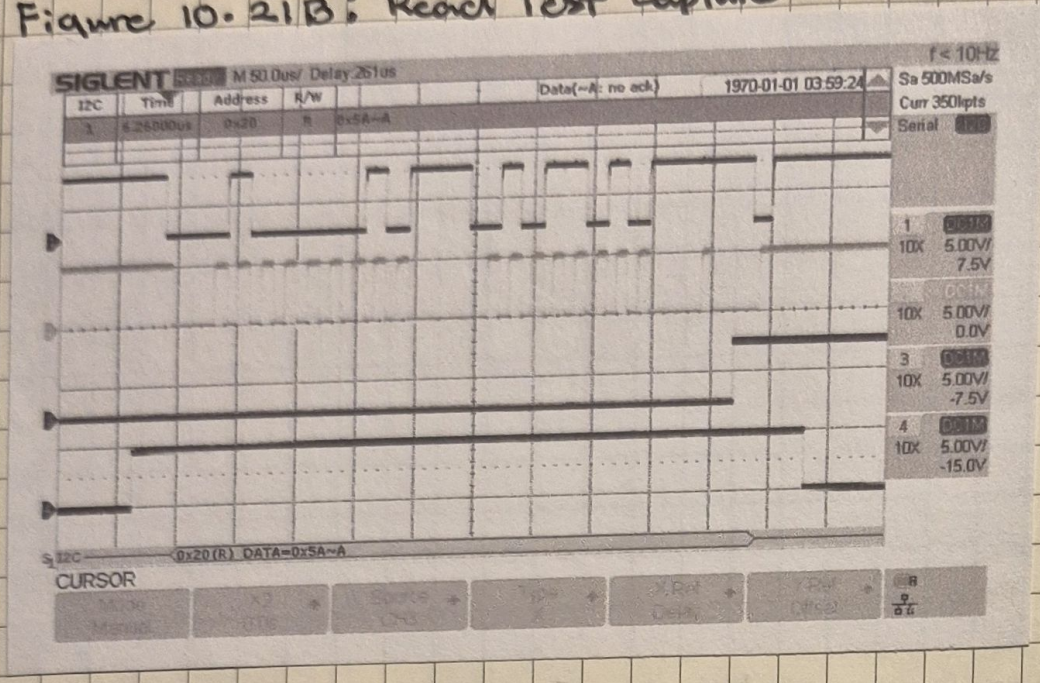
Figure 10.21A: Write Test Capture



Experiment 10: I2C

In The Lab:

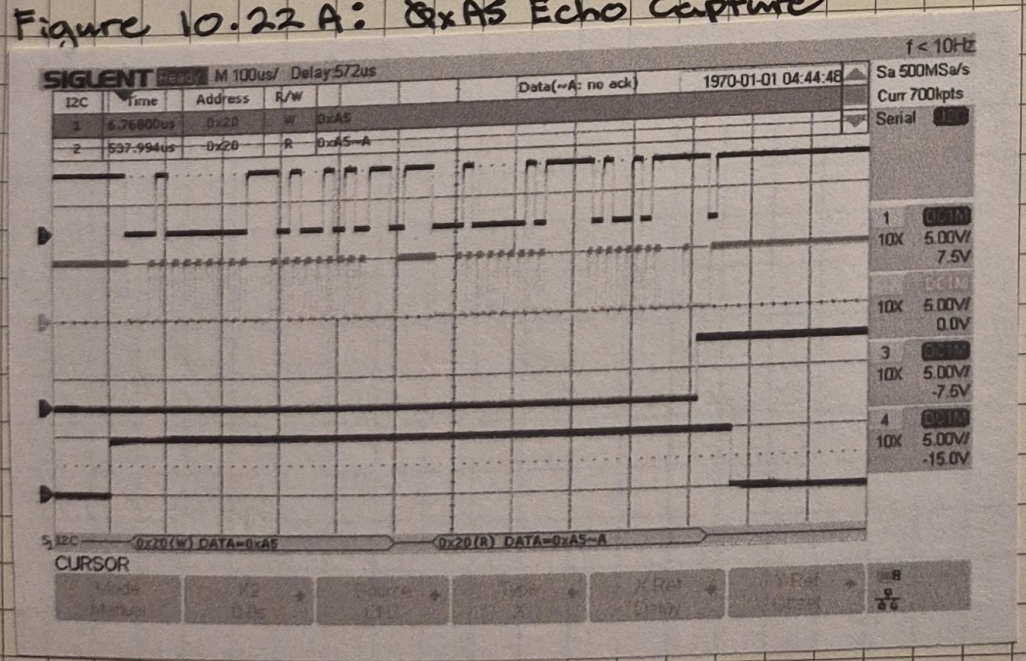
Figure 10.21B: Read Test Capture



3.) Add the register-pointer interface. A slave write begins with a pointer byte. the pointer selects a logical register; optional following bytes write that register. A later master read returns data from the selected register. Retain the selected pointer after STOP sequences. For an initial diagnostic, temporarily echo the selected pointer on a read. Test at least two different values such as 0xA5 and 0x3C. Then implement the following application register maps:

Pointer	Function	Direction	Data
0x00	PORTB switches	Read	One byte, RB5:RB0
0x01	DLG7137 display	Write	One 7-bit ASCII character
0x02	PIC ADC at AN2	Read	Two bytes, high then low
0x04	Device status, including I²C status	Read	One byte

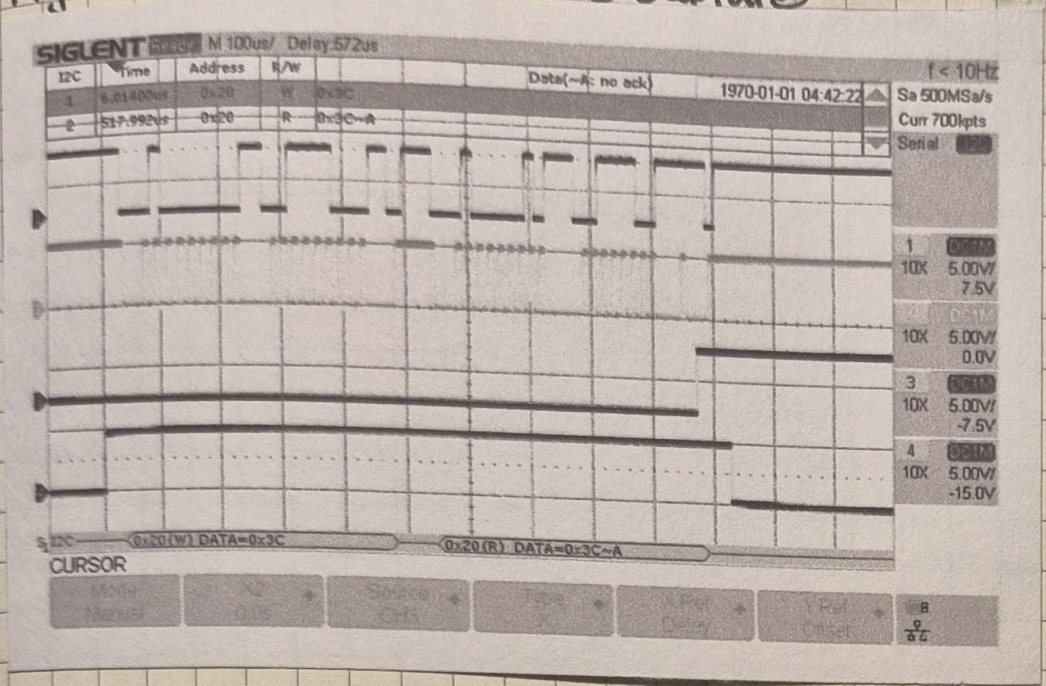
Figure 10.22A: 0xA5 Echo Capture



Experiment 10: I2C

In The Lab:

Figure 10.22 B: 0x3C Echo Capture



4.) Read switches and status. Connect six DIP switches from slave RB0 - RB5 to GND, ~~with the wiper on RA2/AN2 (pin 4)~~ HW C. Configure Enable suitable pull-ups so an open switch reads 1 and closed switch reads 0. Reserve RB6 and RB7 for PICkit programming. Return only bits 5:0 for pointer. Add pointer 0x04 as a one-byte device-status register. Include I2C-related status or fault information, such as MSSP receive overflow or write collision. Choose your own status encoding and handling; document what each value means and how it is cleared. You may also report other useful conditions, such as ADC conversion busy/ready state, invalid data, or application faults. Read the status register after normal transaction and explain the result.

```
Select PORTB: START → 0x40 → ACK → 0x00 → ACK → STOP
Read PORTB:   START → 0x41 → ACK → switch byte → Master NACK → STOP

Select status: START → 0x40 → ACK → 0x04 → ACK → STOP
Read status:   START → 0x41 → ACK → status byte → Master NACK → STOP
```

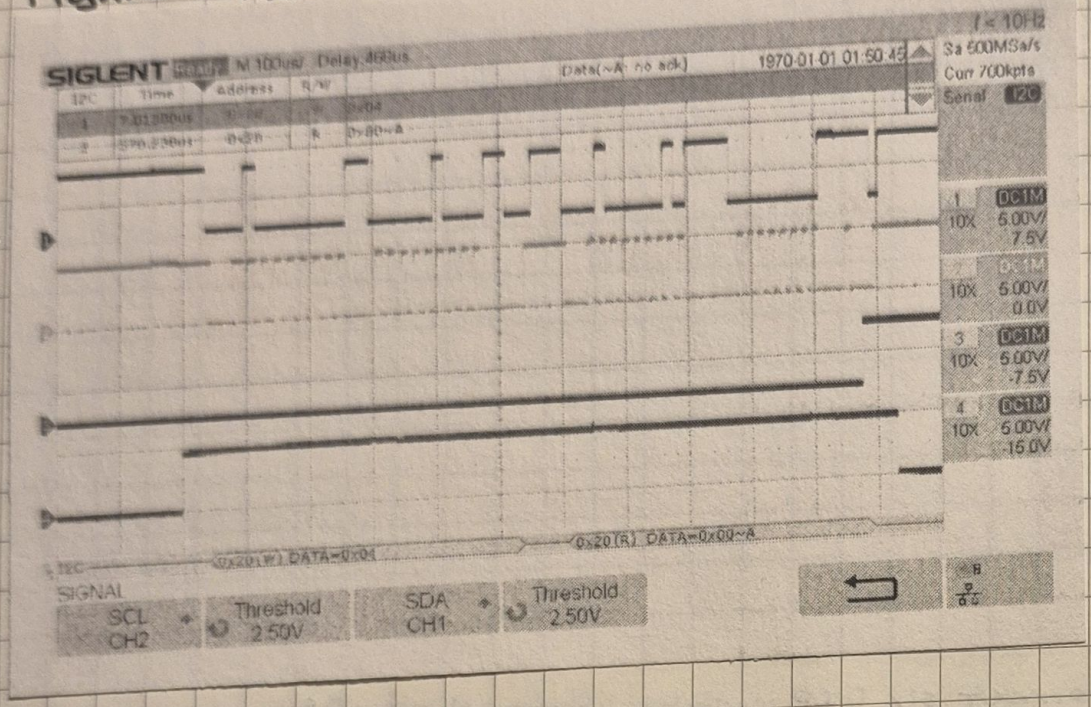
Table 10.5: Measured Switch Data

Switch positions	*Binary (RB5–RB0)*	*Hex data*
All open	111111	0x3F
All closed	000000	0x00
Only RB0 closed	111110	0x3E
Only RB5 closed	011111	0x1F
Only RB0 open	000001	0x01

Experiment 10: I2C

In The Lab:

Figure 10.23: Status Data Capture



Clock stretching experiment. After the PORTB read works normally, add a short inline delay in the slave's PORTB read path, while its MSSP hardware is holding SCL Low and before it releases SCL Low and before it releases SCL to send the data byte. Compare a normal read with several progressively longer delays: for example: 0, 100µs, 1ms, 10ms. Calculate or measure the actual delay. Do not assume that a software delay produces clock stretching unless the scope shows SCL Low.

Table 10.6: Clock-Stretching Data

Target delay	*Measured SCL LOW*	*Successful Read?*
0 µs	5 µs	Yes
100 µs	180 µs	Yes
1 ms	1.08 ms	Yes
10 ms	10.86 ms	Yes

During this experiment, clock stretching was tested using 100µs, 1ms, and 10ms delays. The oscilloscope showed that SCL remained Low longer as the delay increased, while the master continued to receive the DIP switch data successfully.

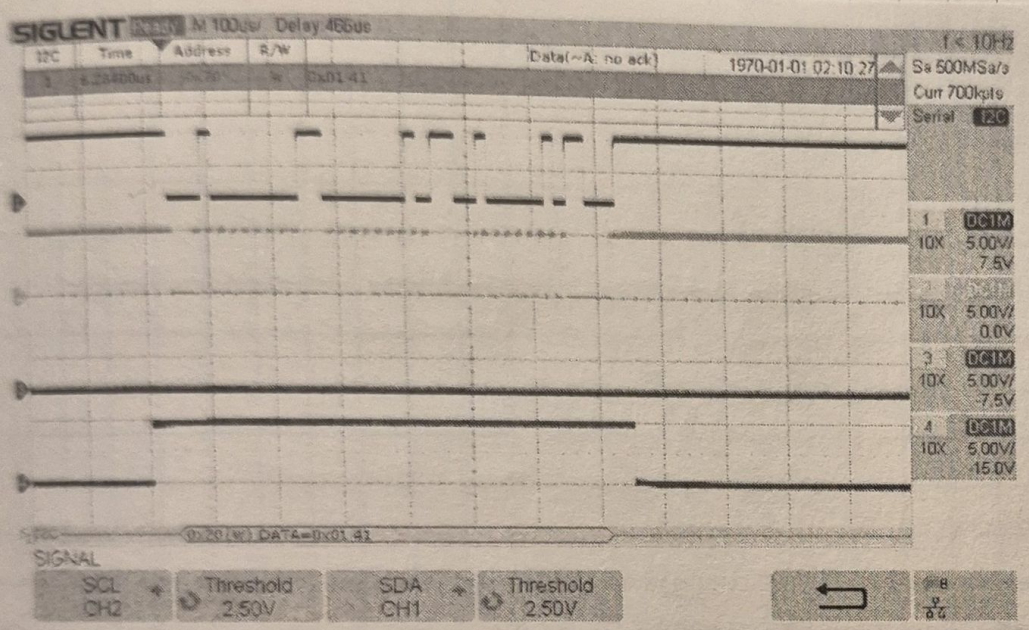
Experiment 10: I2C

In The Lab:

5.) Write a character to the DLG7137. Use the digital display as it accepts 7-bit ASCII; the slave must drive its data lines and pulse it active-low wiring. Write to pointer 0x01 followed by ASCII 0x41 to display "A":

```
START → 0x40 → ACK → 0x01 → ACK → 0x41 → ACK → STOP
```

Figure 10.24: Pointer and ASCII Capture



6.) Read the slave's ADC. Connect a potentiometer between the slave's VDD and GND, with the wiper on RA2/AN2 (pin 4). Configure the PIC's 10-bit ADC, VDD/Vss ref, and the right justified result. Use a timer to sample periodically without waiting in a long blocking loop. Pointer 0x02 returns a consistent two-byte sample: the upper two ADC bits and the lower eight bits. Store conversions in RAM and ensure that I2C read byte cannot come from different ADC conversions.

```
Select ADC: START → 0x40 → ACK → 0x02 → ACK → STOP
Read ADC:   START → 0x41 → ACK → ADC high → Master ACK → ADC low → Master NACK → STOP
```

Combine the received bytes and estimate the input voltage:

$ADC_{10} = ((high \,\&\, 0x03) << 8) \mid Low$

$V_{AN2} \approx (ADC_{10} / 1023) \cdot V_{DD}$

Table 10.7: Measured ADC Data

Input Voltage	*ADC Value*	*High Byte*	*Low Byte*
0 V	0	0x00	0x00
1.26 V	256	0x00	0xFF
2.49 V	512	0x02	0x01
3.80 V	768	0x03	0x01
4.96 V	1023	0x03	0xFF

TJR 11-3-26

Experiment 10: I2C

Part 4: UART-to-I2C Bridge

Goals:

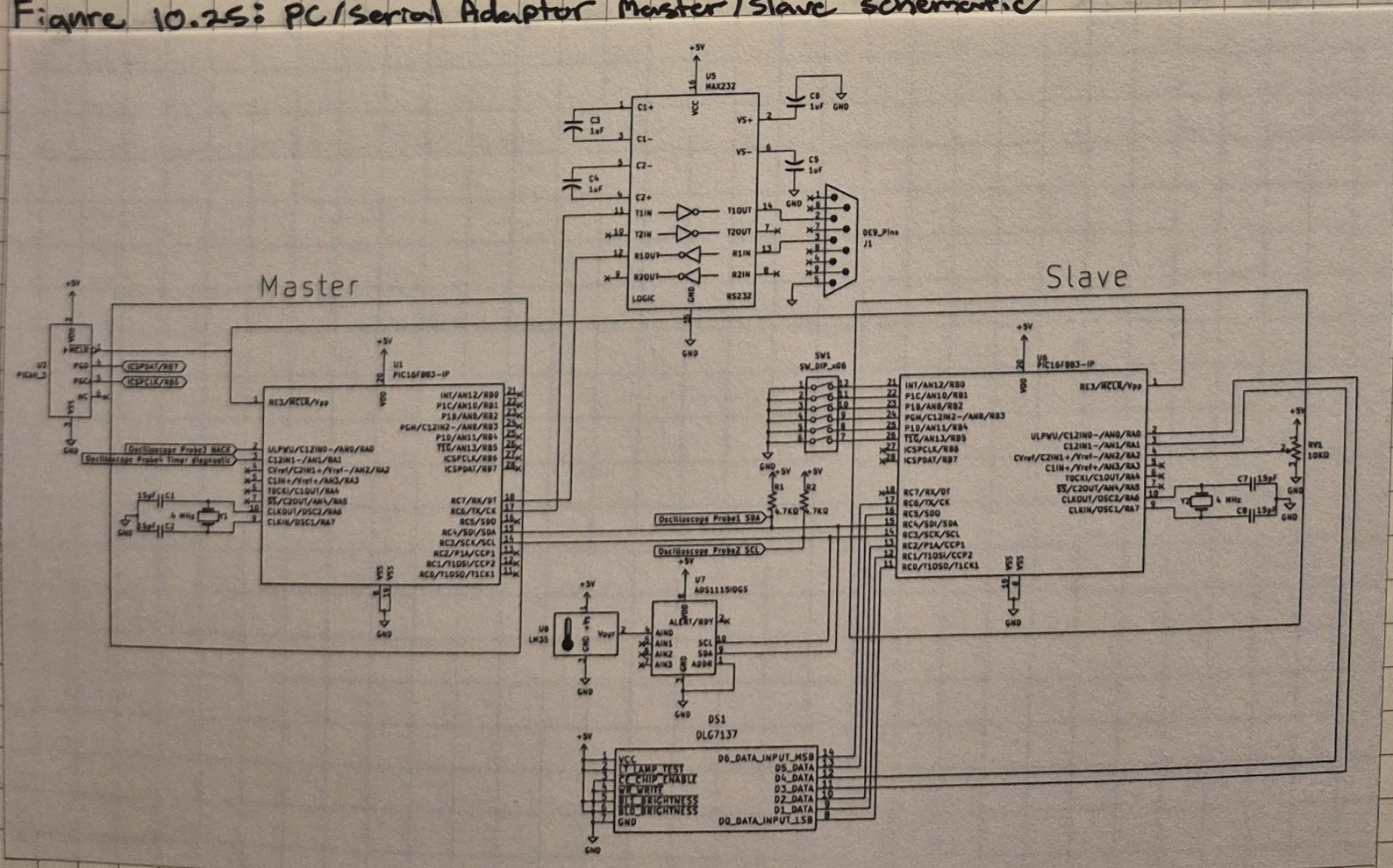
Turn the master PIC16F883 into a generic PC/UART-to-I2C bridge controlled by a student-built GUI. The GUI sends custom byte sequences, read responses, discovers I2C devices, and records communication. Test it with the Part 3 PIC slave and the Part 2 ADS1115. Use the 4 MHz, 100 kHz I2C, and 9600 8N1 UART with a MAX232 or equivalent RS232/TTL interface. Support one host request at a time and ordinary 7-bit addresses. Seperate STOP-terminated writes and reads are sufficient. Repeated START, general call, 10-bit addresses, and multi-master operation are not required.

Before Lab:

Prepare:

- An updated schematic showing the PC serial adapter, MAX232, master PIC, both existing I2C devices, common ground, power, pull-ups, and compatible logic levels. Account for pull-ups on the ADS1115 breakout.
- Flowcharts for receiving a UART frame, validating the request, performing an I2C transaction, and returning the result. Document the EUSART and MSSP registers you use.
- A command/response packet table and test cases for ACK, NACK, malformed requests, and ~~returning the result~~ transfer-length limit.
- A GUI plan covering hex/text entry, saved transmissions, live UART/I2C logs, a 128-address scan, and automatic bridge identification/reconnection.

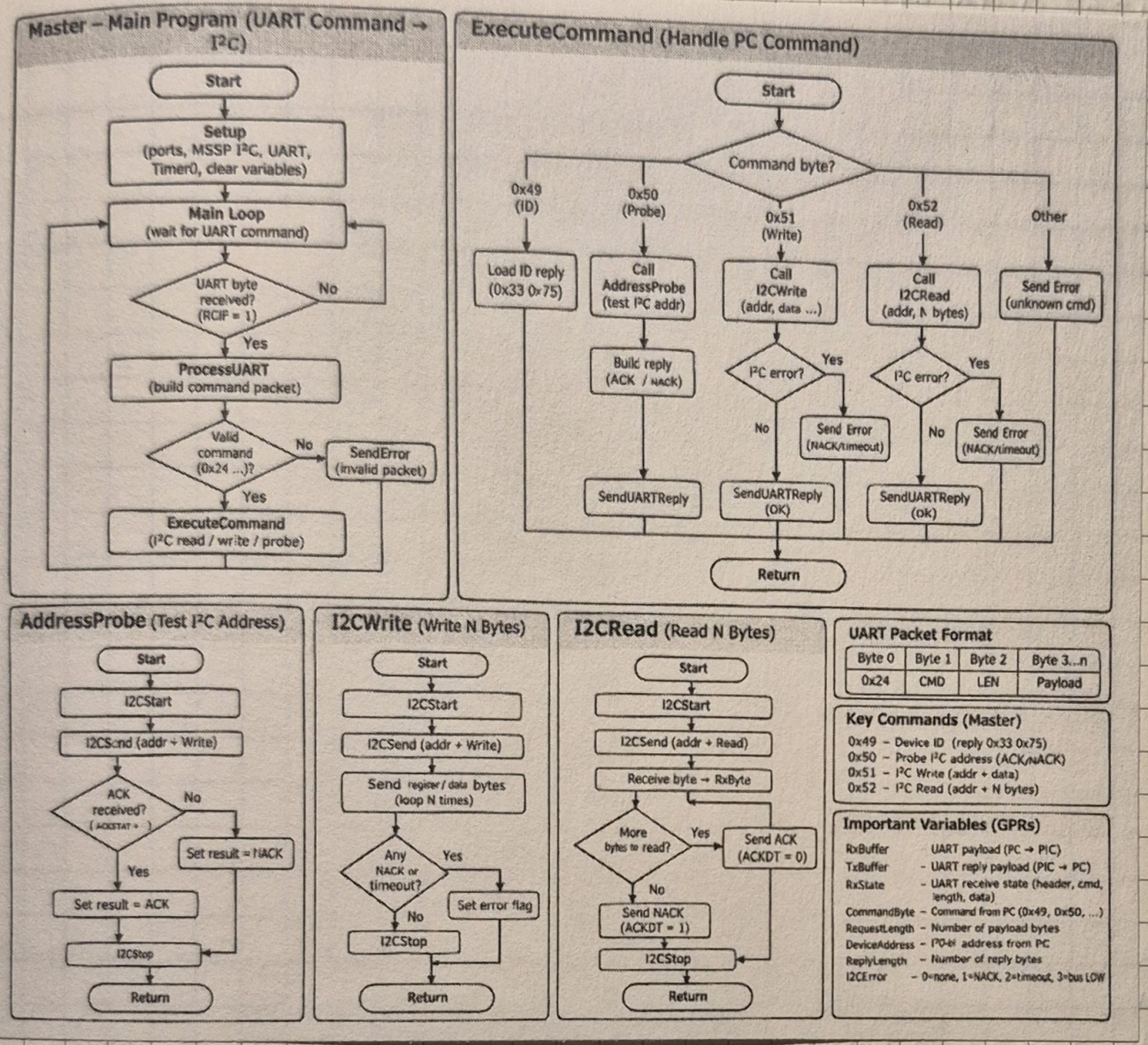
Figure 10.25: PC/Serial Adapter Master/Slave schematic



Experiment 10: I2C

Before Lab:

Figure 10.26: Flowchart P+4 Master



The slave code from Part 3 remains the same. For the full code, please refer to this link: https://github.com/hunterclezie-pixel/Lab-10---I2C-Communication.git

UART packet format and error packets:

Uart packets contain binary bytes, shown below in hexadecimal. Each frame is 24, one command byte, a one-byte payload length, then that many payload bytes. Implement a minimum accepted payload of 24 bytes. The bridge must reject invalid lengths without overflowing its buffer or initiating I2C traffic.

Command byte	Operation	Host payload	Successful reply payload
49	Identity	None	33 75
58	Address probe	AA	00
57	Raw write	AA DD...	00
52	Raw read	AA NN	00 DD...
45	Error response only	None	Two error bytes (below)

Experiment 10: I2C

UART packet format and error packets:

Raw writes contain 1-23 data bytes; raw read request 1-23 bytes. Successful replies include the full 24, command, length, and payload. On failure, return one error packet instead of a successful command reply. The bridge sends the host's write bytes unchanged and does not interpret I2C device registers. A pointer writes a subsequent read and seperate requests. Preserve the Lab 8 UART framing and overrun error packets. Add seperate I2C and bridge-validation error catigories without reusing those codes. Every error packet has 5 bytes: 24 45 02, error category, detail.

Test	UART TX	UART RX	Expected I²C transaction
Probe ADS1115	24 50 01 48	24 50 01 00	START, 90, ACK, STOP
Select Config	24 57 02 48 01	24 57 01 00	START, 90, ACK, 01, ACK, STOP
Read Config	24 52 02 48 02	24 52 03 00 HH LL	START, 91, ACK, HH, ACK, LL, NACK, STOP
Select Conversion	24 57 02 48 00	24 57 01 00	START, 90, ACK, 00, ACK, STOP
Read Conversion	24 52 02 48 02	24 52 03 00 HH LL	START, 91, ACK, HH, ACK, LL, NACK, STOP

I²C error detail (SS)	Meaning	Bridge error detail (EE)	Meaning
01	Address/data NACK	01	Invalid length or count
02	I²C timeout	02	Unknown command
03	Bus LOW / not idle	03	Invalid 7-bit address
04	SCL held LOW		

Preserve Lab 8's UART receiver recovery behavior: discard a damaged, partial packet, clear the receive fault, and accept the next valid request. If an I2C operation fails, stop/clean up the bus when possible, report the error and allow a later request. A request produces one response: either the normal success or the error packet.

Legend for all transactions

Symbol	Meaning
AA	Seven-bit I²C device address
NN	Number of bytes requested by a read
SS	I²C error detail
EE	Bridge error detail
VV	PORTB switch byte
HH	High result byte
LL	Low result byte
DD	One variable data byte
× N	Repeat the preceding byte/sequence N times (not transmitted)
START / STOP	I²C bus conditions
ACK / NACK	I²C ninth-bit responses, not data bytes

Experiment 10: I2C

Expected UART TX/RX and I2C Transactions:

UART TX: Host → bridge. UART RX: bridge → host. The Part 3 PIC slave has assigned 7-bit address 20, so its I2C address byte are 40 for write and 41 for read. The examples using using 21 as an absent address assume no device responds at 21 on your bus; choose another unused ordinary address if necessary.

Complete error packet	Meaning
24 45 02 01 00	UART framing error, retained from Lab 08
24 45 02 02 00	UART receive overrun, retained from Lab 08
24 45 02 04 SS	New I²C error; SS identifies the cause
24 45 02 05 EE	New bridge request/validation error; EE identifies the cause

Each pointer writes and reads a seperate UART Request and a seperate I2C transaction with stop between them.
For the ADS1115 examples, 48 is a sample 7-bit address, with I2C address byte 90 (write) and 91 (read). Substitute your verified Part 2 address and its corresponding address bytes.

Test	UART TX	UART RX	Expected I²C transaction
Identify	24 49 00	24 49 02 33 75	None
Probe PIC, ACK	24 50 01 20	24 50 01 00	START, 40, ACK, STOP
Probe absent 21	24 50 01 21	24 45 02 04 01	START, 42, NACK, STOP
Display character 41	24 57 03 20 01 41	24 57 01 00	START, 40, ACK, 01, ACK, 41, ACK, STOP
Select PORTB	24 57 02 20 00	24 57 01 00	START, 40, ACK, 00, ACK, STOP
Read PORTB	24 52 02 20 01	24 52 02 00 VV	START, 41, ACK, VV, NACK, STOP
Select PIC ADC	24 57 02 20 02	24 57 01 00	START, 40, ACK, 02, ACK, STOP
Read PIC ADC	24 52 02 20 02	24 52 03 00 HH LL	START, 41, ACK, HH, ACK, LL, NACK, STOP

Errors and transfer boundaries

Test	UART TX	UART RX	Expected I²C transaction
Write NACK, absent 21	24 57 02 21 00	24 45 02 04 01	START, 42, NACK, STOP
Read NACK, absent 21	24 52 02 21 01	24 45 02 04 01	START, 43, NACK, STOP
Bad probe length	24 50 00	24 45 02 05 01	None
Unknown command	24 51 00	24 45 02 05 02	None
Invalid address	24 50 01 00	24 45 02 05 03	None
Zero-byte read	24 52 02 20 00	24 45 02 05 01	None
Address-only write	24 57 01 20	24 45 02 05 01	None
Invalid 24-byte read	24 52 02 20 18	24 45 02 05 01	None
Write with 24 data bytes (oversized)	24 57 19 20 01 + 41 × 23	24 45 02 05 01	None
23-byte read (device supports it)	24 52 02 20 17	24 52 18 00 + DD × 23	START, 41, ACK, (DD, ACK) × 22, DD, NACK, STOP
23-byte write (device supports it)	24 57 18 20 01 + 41 × 21 + 42	24 57 01 00	START, 40, ACK, 01, ACK, (41, ACK) × 21, 42, ACK, STOP

Experiment 10: I2C

In The Lab:

1.) Start the host GUI and UART link. Extend the Lab 8 host application or create a new GUI with serial-port communication and raw hex transmission.
2.) Address probes. Implement probe command 50. Probe the slave at 20 and an address you have confirmed is unoccupied. Confirm ACK/NACK with decoded start, address, and stop on the scope.
3.) Generic write/read. Implement write command 57 and read command 52 using existing I2C transaction routine and bounded buffers.
4.) Second I2C device. Connect the ADS1115 and PIC slave atthe same time.
5.) Use the GUI to try zero-length, one-byte, ordinary, max-length, and oversized request, including a zero-byte read, an address-only write, a 23-byte read request (17), a 24-byte read request (18), and a write with 24 data bytes (25-byte payload, above the bridge's limit).
6.) Complete the host GUI. Add the remaining features: Use the GUI to repeat the PIC and ADS1115 transactions, scan all addresses, save and replay commands, then test automatic connection and recovery.

Figure 10.27: Host GUI

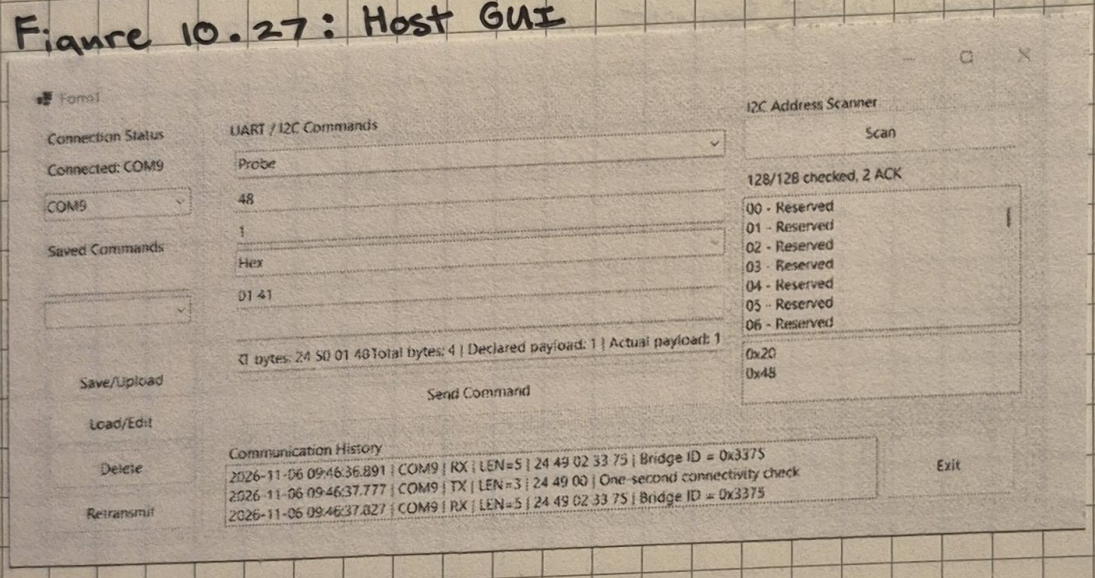
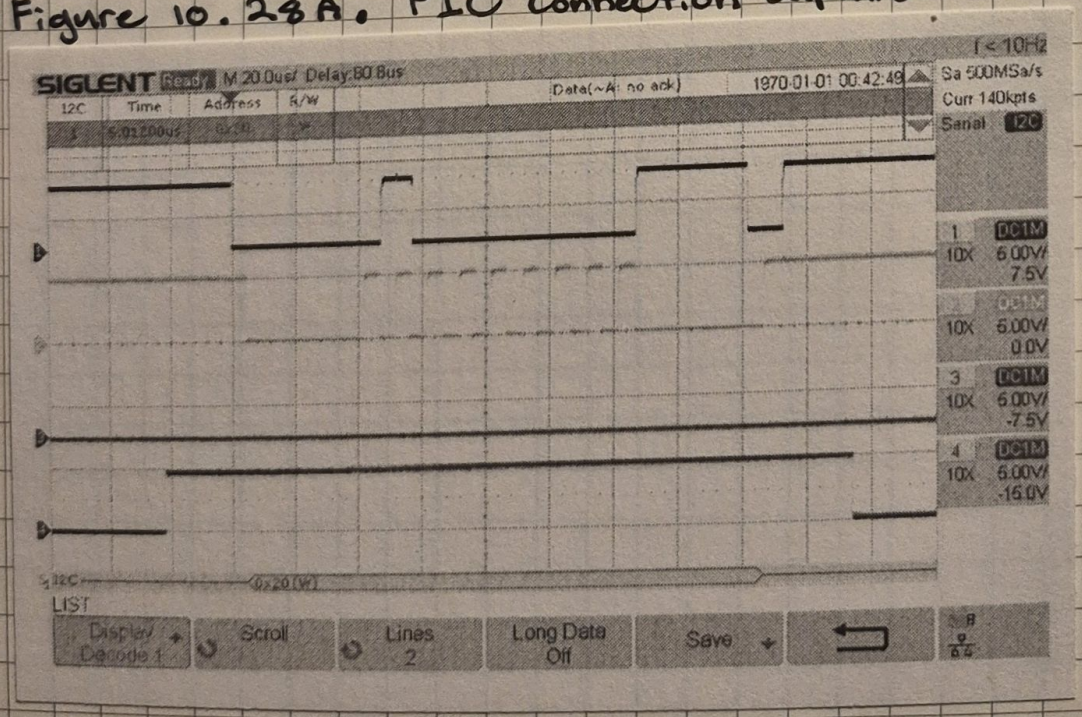


Figure 10.28A: PIC Connection Capture



Experiment 10: I2C

In The Lab:

Figure 10.28 B: ADS1115 connection Capture

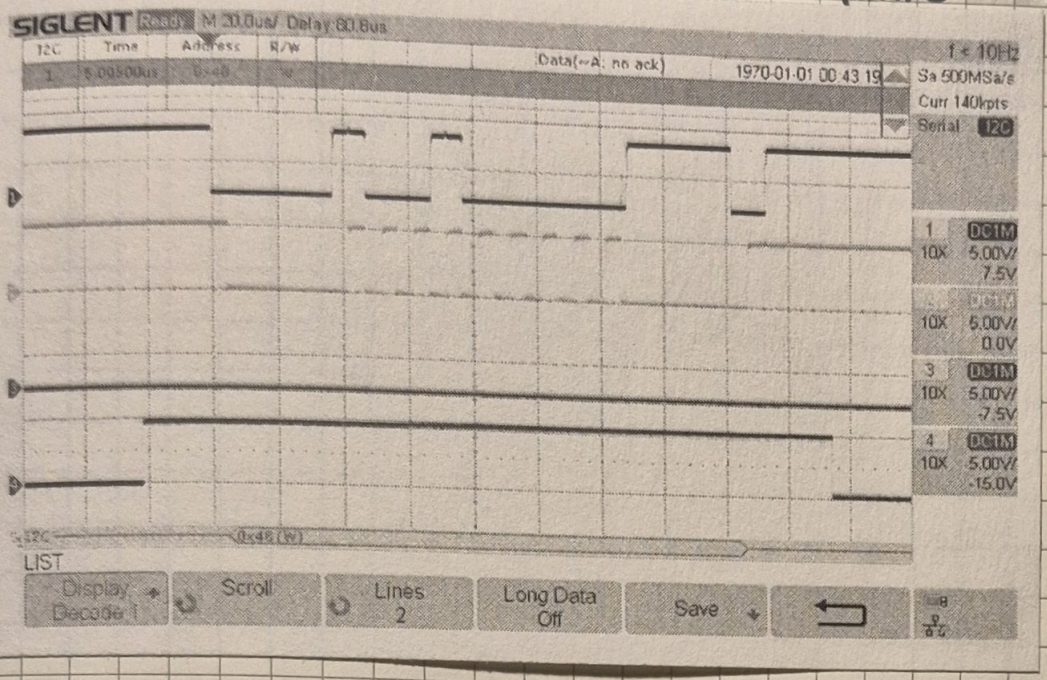


Figure 10.28 C: No Connection Address Capture

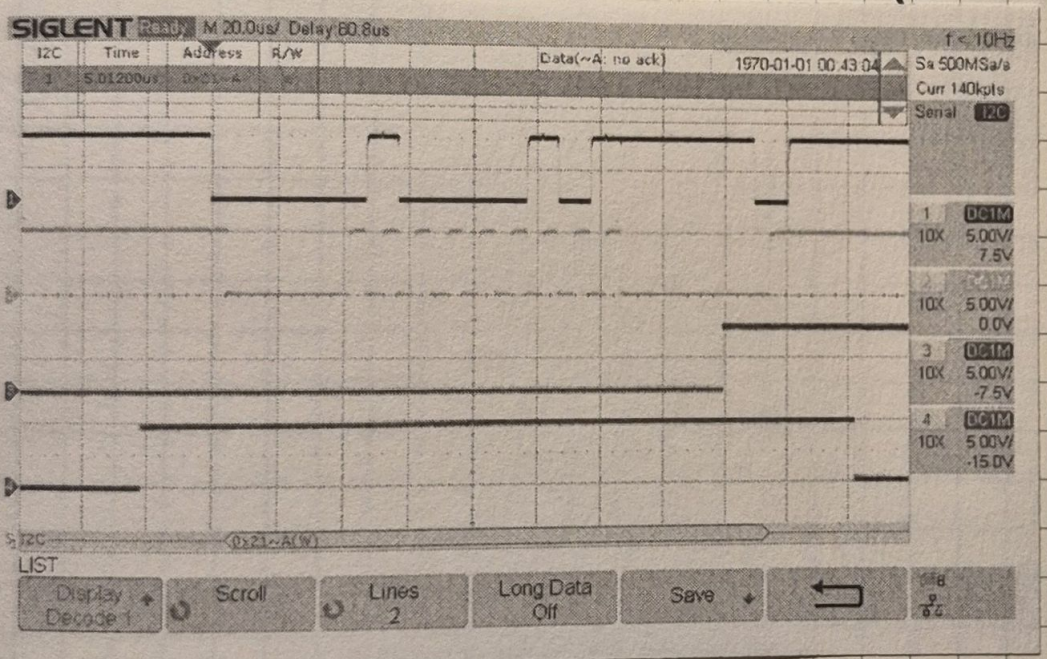
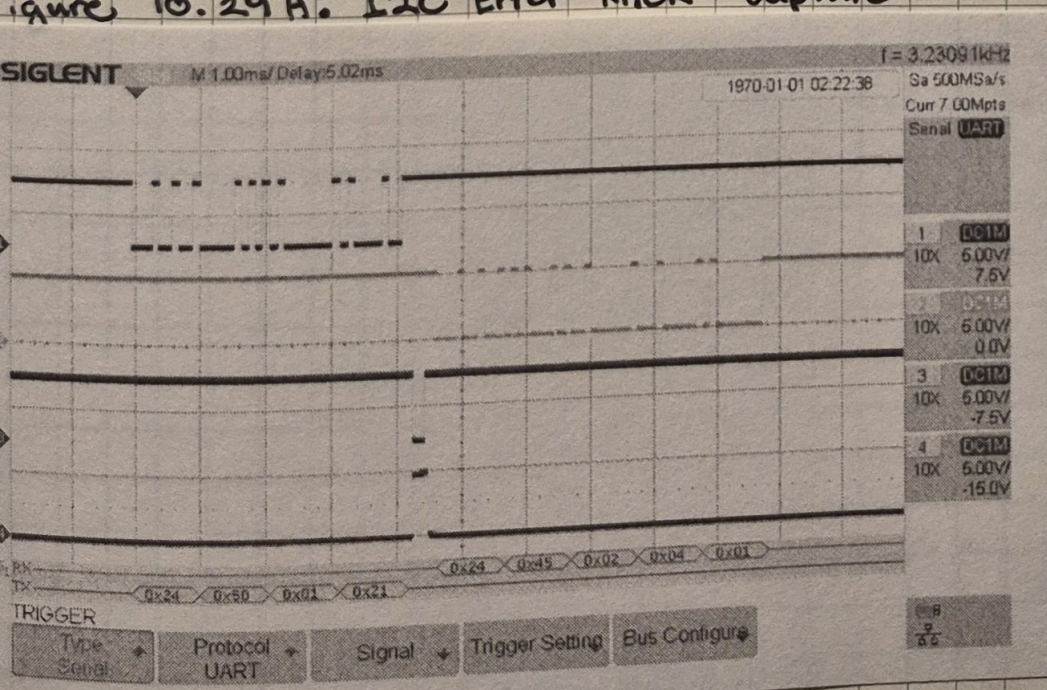


Figure 10.29 A: I2C Error NACK Capture



Experiment 10: I2C

In The Lab:

Figure 10.29B: Invalid Length Error Capture

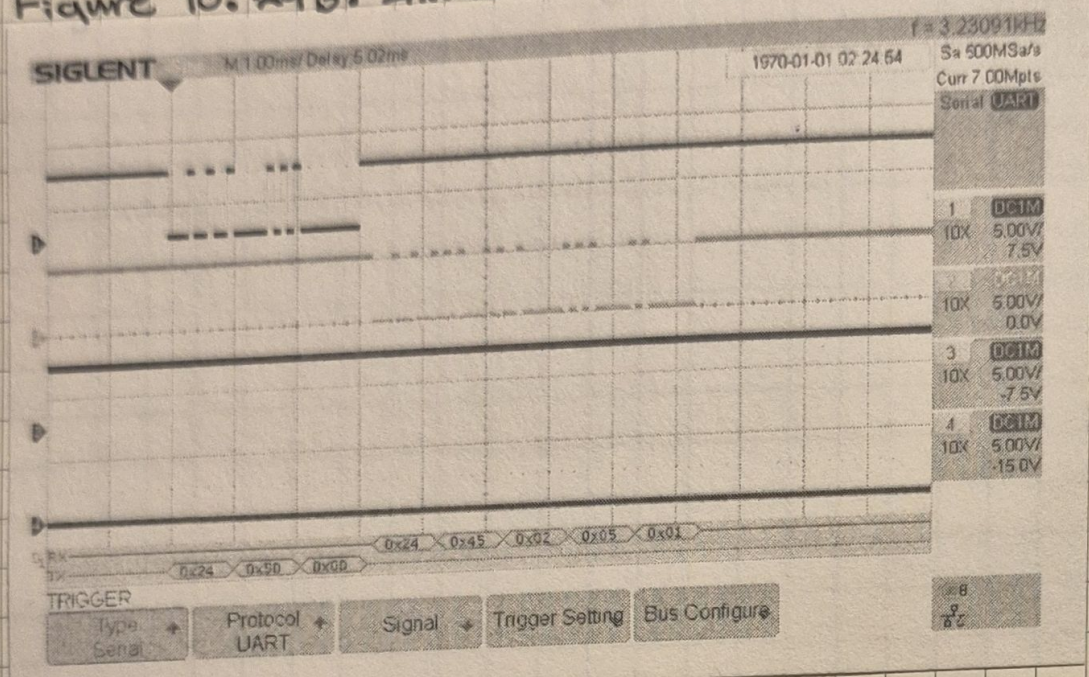
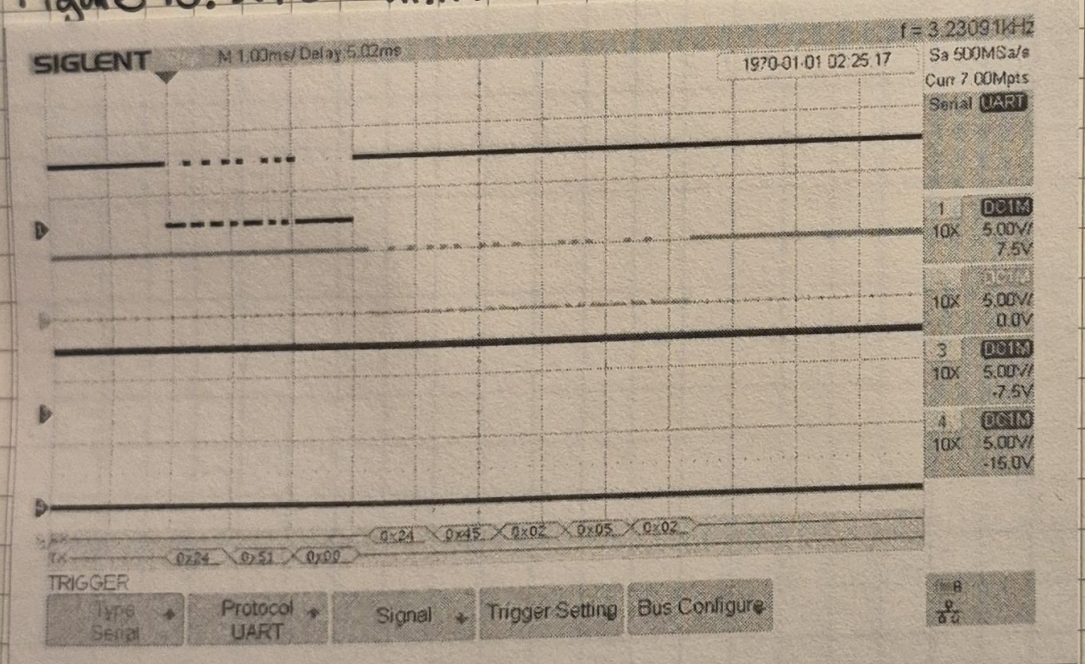


Figure 10.29C: Unknown Command Error Capture



TJR
11-6-26

Experiment 10: I2C

Conclusion:

Throughout this lab, I2C communication connected multiple devices to the PIC16F883 over a two-wire bus. In Part 1, the PIC was configured as an I2C master at 100KHz, and the SCL and SDA signals were measured to observe START, STOP, and ACK/NACK conditions. We Different pull-up resistor values were used to understand their effects on rise time, voltage, and current. In Part 2, an ADS1115 ADC was connected as an I2C slave, allowing the PIC to identify its address, configured ~~as an~~ its registers, and read 16-bits ADC measurements. In Part 3, a second PIC16F883 was configured as an I2C slave, allowing the master to read DIP-Switch inputs, send characters to a display, and retrieve 10-bit ADC readings. Clock stretching was also ~~integrated with I2C to bridge a computer and multiple~~ investigated to understand how a slave can delay communication. In Part 4, UART communication was integrated with I2C to bridge a computer and multiple I2C devices. A C# graphical application was developed to send commands, scan I2C addresses, display received data, and record communication errors while automatically managing the connections. Overall, this lab showed how I2C, UART, and software communication protocols can work together to create a flexible system for controlling and monitoring multiple devices.